Approximation and Learning-based Algorithms for Influence Maximization in Multilayer Social Networks

Xueqin Chang
Zhejiang University
Hangzhou, China
changxq@zju.edu.cn

Ruize Liu
Zhejiang University
Hangzhou, China
lrz@zju.edu.cn

Qing Liu*
Zhejiang University
Hangzhou, China
qingliucs@zju.edu.cn

Baihua Zheng
Singapore Management University
Singapore
bhzheng@smu.edu.sg

Yunjun Gao
Zhejiang University
Hangzhou, China
gaoyj@zju.edu.cn

Abstract

Motivated by the observation that users in the real world often engage across multiple social networks simultaneously, we study the problem of influence maximization in multilayer social networks (Mlim), aiming to select a small set of nodes that maximizes the total influence spread across all layers. To this end, we introduce a hybrid propagation model that jointly captures layer-specific diffusion dynamics and probabilistic cross-layer propagation. Based on this model, we formally define the Mlim problem and establish its **NP**-hardness, monotonicity, and submodularity. To address the Mlim problem, we first propose a greedy baseline Mlim-Greedy, which achieves a $(1-1/e)$ approximation. Since exact influence computation in Mlim-Greedy is #**P**-hard, we propose STARIM, a scalable algorithm with layer-weighted influence sampling that guarantees a $(1-1/e-\epsilon)$ approximation. To further enhance efficiency, we design LGQIM, a two-stage framework where multilayer representation learning predicts influence spread from network structure, enabling deep reinforcement learning for adaptive seed selection. Extensive experiments on nine real-world datasets demonstrate that **(1)** STARIM is up to 2 orders of magnitude faster than the baselines while yielding 10%-30% improvement in influence spread, and **(2)** LGQIM further achieves an average 10× speedup over STARIM while maintaining comparable influence spread.

CCS Concepts

• **Information systems** → **Social networks**; • **Theory of computation** → **Approximation algorithms analysis**; • **Computing methodologies** → **Machine learning algorithms**.

Keywords

Influence maximization; multilayer social networks; theoretical analysis; graph embedding; deep reinforcement learning

ACM Reference Format:

Xueqin Chang, Ruize Liu, Qing Liu, Baihua Zheng, and Yunjun Gao. 2026. Approximation and Learning-based Algorithms for Influence Maximization in Multilayer Social Networks. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026), August 9–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3770855.3817870

*Corresponding author.

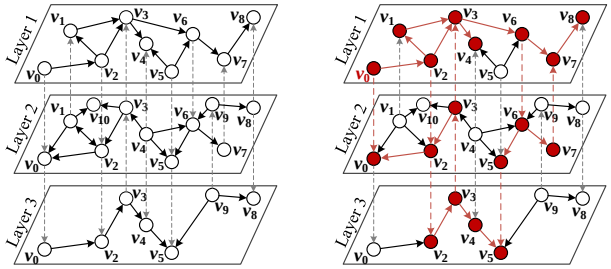
This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD 2026, Jeju Island, Republic of Korea.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817870

1 Introduction

Influence maximization (IM) [28] aims to identify a set of k seed nodes that serve as the sources of information diffusion such that the expected number of influenced nodes is maximized under a specified diffusion model. This fundamental problem in social network analysis has wide-ranging applications, including viral marketing [16], diffusion control [61], and social recommendation [63]. Traditional IM research primarily focuses on single-layer networks [28, 35]. In practice, however, users typically participate in multiple social networks simultaneously, with information flowing across different platforms [22]. For instance, GWI reports that users engage with an average of 6.84 social platforms monthly [13], and recent research shows that multi-platform marketing achieves 2–5% higher sales than single-platform strategies [62]. These observations motivate the study of IM in multilayer social networks [1, 52], which more accurately capture real-world diffusion dynamics and enable applications such as cross-platform marketing [2, 26, 30, 46, 65].

Existing diffusion models for IM in multilayer social networks [15, 26, 27, 46, 47, 50, 64] typically assume *uniform* propagation patterns across all layers. However, influence propagation varies significantly across different platforms in reality. For example, information often diffuses through friend connections on Facebook [44], while it mainly spreads via broadcasting mechanisms on Twitter [3]. Additionally, several prior works [26, 30, 46, 64] assume that influenced users automatically share content with all their friends on other networks, which is unrealistic. For example, users sharing entertainment content on TikTok rarely propagate the same content on professional platforms such as LinkedIn. To address these limitations, we introduce a *hybrid propagation model* where each layer follows a distinct platform-specific propagation mechanism, and cross-layer propagation is modeled probabilistically to capture users' selective sharing behaviors across different platforms.

Multilayer Influence Maximization Problem. Based on the proposed hybrid propagation model, we revisit the MultiLayer Influence Maximization (Mlim) problem, defined as follows. Given a multilayer graph $G(V, E, L)$ and a budget k, the Mlim problem aims to identify a seed node set $S \subset V$ with $|S| = k$ that maximizes the



(a) A multilayer social network G (b) Influence spread in G with S={$v_{0,1}$}

Figure 1: A motivating example

expected influence spread, i.e., the expected total number of nodes influenced by S across all layers under the hybrid propagation model. Figure 1 illustrates influence spread in a multilayer social network G under the hybrid propagation model. With a budget of $k = 1$, the optimal seed node set is $S = \{v_0^{(1)}\}$, achieving a maximum influence spread of 18, representing the total cross-platform exposure [7].

Applications. Companies increasingly leverage multiple social platforms for product promotion [60]. For instance, brands often collaborate with influencers across platforms such as Instagram, TikTok, and YouTube in integrated marketing campaigns [20]. Research shows that multi-platform campaigns significantly boost user engagement, as cross-platform exposure strengthens purchase intent [62]. By applying Mlim, companies can identify influential seed users across multiple networks, thereby maximizing cross-platform exposure and improving marketing effectiveness.

Challenges and Solutions. Existing methods for IM in single-layer networks cannot be directly applied to the Mlim problem, as they employ a single propagation model across the entire network and fail to capture heterogeneous propagation dynamics both across and within different layers [18, 23, 54]. To fill this gap, several approaches have been proposed for multilayer IM [1, 30, 46, 50], yet they still face significant limitations. First, approximation algorithms [30, 66] offer theoretical guarantees but incur prohibitive computational costs due to Monte Carlo simulations [28], making them impractical for large-scale networks. Second, heuristic methods [26, 50, 52, 67] are more scalable but lack guarantees, often yielding suboptimal solutions. Third, learning-based approaches [15, 27, 47, 64] rely on static structural features and overlook the dynamic nature of influence propagation. Thus, there is a clear need for *new approximation and learning-based algorithms* that can effectively and efficiently address the Mlim problem.

We prove that Mlim under the hybrid propagation model is **NP**-hard, monotone, and submodular. Based on these properties, we first propose Mlim-Greedy, which extends the classical greedy algorithm [28] to multilayer networks by iteratively selecting nodes to maximize influence spread across all layers, achieving a $(1 - 1/e)$-approximation. Since computing exact influence spread in Mlim-Greedy is #**P**-hard and impractical for large networks, we propose STARIM, a scalable trial-and-error algorithm. To this end, we introduce a novel *Multilayer Reverse Influence Sampling* (*MRIS*) technique. Unlike classical RIS, which is designed for single-layer networks and uniformly samples source nodes under a *single* propagation model, *MRIS* employs *layer-weighted influence sampling* by introducing *multilayer reverse reachable* (M-RR) sets to capture cross-layer propagation dynamics. Specifically, M-RR sets are generated by sampling layers proportionally to their sizes, and then performing reverse propagation across both intralayer and interlayer edges according to each layer's propagation model. Based on generated M-RR sets, we design an unbiased estimator for influence spread in multilayer networks. The key innovation lies in layer-specific influence estimation by explicitly tracking the layer associated with each source node, and then aggregating these layer-wise estimates to obtain an unbiased estimate of total influence across all layers. Leveraging *MRIS* and this estimator, STARIM selects seed nodes with a $(1 - 1/e - \epsilon)$-approximation with high probability.

While STARIM provides theoretical guarantees for the Mlim, generating M-RR sets requires multiple graph traversals, which becomes prohibitively expensive on large-scale graphs and limits *real-time applicability*. To overcome this limitation, we propose LGQIM, an efficient learning-based framework with a two-stage design: multilayer influence prediction and adaptive seed selection.

In the first stage, LGQIM employs supervised learning to predict node influence scores based on multilayer structural features. Specifically, it learns specialized embeddings that capture both intralayer and interlayer structures while encoding cross-layer propagation dynamics. This enables efficient influence prediction without repeated graph traversals for M-RR set generation. In the second stage, recognizing that optimal seed selection is fundamentally a combinatorial problem where the marginal gain of each node depends on previously selected seeds, LGQIM formulates the process as a Markov Decision Process and applies deep reinforcement learning to learn an optimal selection policy. In this formulation, the state represents the current seed node set, an action corresponds to adding a new seed node, and the reward reflects the marginal influence gain. Using the predicted influence scores from the first stage, along with multilayer node features and adaptive marginal gains, as the state representation, LGQIM learns a policy to sequentially construct high-quality seed node sets. At test time, given a multilayer graph and a budget k, LGQIM first predicts influence scores for all nodes through the learned embeddings, then sequentially constructs the seed set according to the learned selection policy. This design enables LGQIM to achieve superior efficiency for real-time and large-scale applications while maintaining solution quality.

Contributions. Our contributions are summarized as follows.

- We introduce a hybrid propagation model for multilayer networks, formally define the Mlim problem, and prove it is **NP**-hard, monotone, and submodular.
- We propose a greedy baseline Mlim-Greedy and its scalable implementation STARIM via layer-weighted influence sampling, achieving $(1-1/e)$ and $(1-1/e-\epsilon)$ approximations, respectively.
- We develop LGQIM, a two-stage learning framework that integrates multilayer representation learning for influence prediction and deep reinforcement learning for seed selection.
- We conduct extensive experiments on nine real-world multilayer networks to demonstrate the effectiveness, efficiency, and scalability of the proposed algorithms.

Roadmap. Section 2 reviews related work. Section 3 defines the hybrid propagation model and Mlim problem. Section 4 introduces the approximation algorithms. Section 5 presents the learning-based framework. Section 6 reports experiments, and Section 7 concludes the paper. *All omitted proofs are deferred to our technical report [10].*

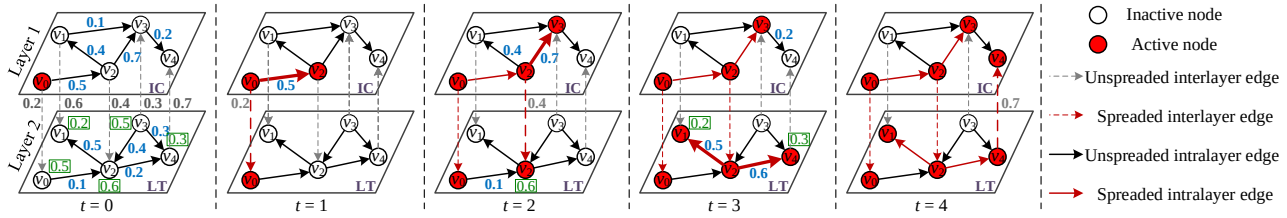


Figure 2: Illustrating influence propagation under *hybrid propagation model*; green numbers besides nodes are activation thresholds θ, blue numbers on the edges are influence weights w

2 Related Work

Influence Maximization in single-layer networks. Kempe et al. [28] first formulated the Influence Maximization (IM) problem, and proposed a greedy algorithm with a $(1-1/e)$-approximation. Subsequent research has developed efficient and scalable IM solutions [6, 18, 19, 24, 54, 56] and explored practical variants of IM [5, 8, 9, 12, 55, 58, 61, 68]. Besides, several learning-based approaches apply reinforcement learning to learn optimal seed selection policies [33, 34, 36, 37, 59], and graph neural networks to generate node embeddings for seed selection [17, 31, 38, 40, 49].

Influence Maximization in multilayer networks. Recent research has extended IM to multilayer networks, with approaches broadly falling into three categories [1]: **(1)** Approximation methods [30, 66] adapt greedy algorithms to multilayer settings. Kuhnle et al. [30] developed the KSN framework that combines single-layer IM algorithms with knapsack optimization techniques. **(2)** Heuristic methods [26, 50, 52, 67] leverage network structural properties for seed selection. CBIM [50] detected communities in each layer and selected seeds using a quota-based strategy guided by edge weights. **(3)** Learning-based methods [15, 27, 47, 64] apply machine learning techniques for seed selection. Keikha et al. [27] selected seed nodes using node feature vectors learned through network embedding. However, approximation methods suffer from high computational cost due to Monte Carlo simulations, heuristic methods lack theoretical guarantees, and learning methods often capture static structure rather than propagation dynamics, yielding suboptimal seed set.

3 Preliminaries

In this section, we introduce the hybrid propagation model and define the Multilayer Influence Maximization (MLIM) problem.

We consider a multilayer network $G(V, E, L)$, where V, E, and $L = \{1, 2, \ldots, l\}$ denote the sets of nodes, edges, and layers, respectively. Each layer $i \in L$ is a directed graph $G_i = (V_i, E_i)$, with $V_i \subseteq V$ and $E_i \subseteq V_i \times V_i$. Interlayer edges between layers i and j ($i, j \in L$, $i \neq j$) are given by $E_{i,j} \subseteq V_i \times V_j$, connecting corresponding nodes across layers. The overall node and edge sets are given by $V = \bigcup_{i \in L} V_i$ and $E = \bigcup_{i \in L} E_i \cup \bigcup_{i \neq j} E_{i,j}$. For a directed edge $e = (u, v) \in E$, u is an in-neighbor of v and v is an out-neighbor of u. We denote the in- and out-neighbor sets of v by $N_{in}(v)$ and $N_{out}(v)$, respectively, and their intralayer counterparts in layer i by $N^i_{in}(v)$ and $N^i_{out}(v)$. Each edge $e = (u, v) \in E$ is associated with an influence weight $w_{u,v} \in [0, 1]$, quantifying the influence of u on v.

3.1 The Hybrid Propagation Model

Building upon the above settings, we introduce the hybrid propagation model, where each layer is assigned a specific propagation model. In this paper, we focus on two widely adopted propagation models: the independent cascade (IC) and linear threshold (LT) models [28]. *Note that the hybrid propagation model is general and can incorporate other propagation models.*

Given a multilayer graph $G(V, E, L)$ and a seed set $S \subset V$, each active on its layer at timestamp 0, influence propagates iteratively across both intralayer and interlayer edges as follows:

- Intralayer propagation. Within each layer $i \in L$, propagation follows the assigned model: (1) Under the IC model, when a node $u^{(i)}$ becomes active at timestamp $t \geq 0$, it has one chance to activate each inactive out-neighbor $v^{(i)}$ in layer i at $t+1$ with probability $w_{u^{(i)},v^{(i)}}$. (2) Under the LT model, each node $v^{(i)}$ is assigned an activation threshold θ^i_v uniformly at random from $[0, 1]$. At timestamp $t > 0$, an inactive node $v^{(i)}$ becomes active iff the total influence from its active in-neighbors in layer i exceeds its threshold: $\sum_{u^{(i)} \in N^i_{in}(v) \cap A_{t-1}} w_{u^{(i)},v^{(i)}} \geq \theta^i_v$, where A_{t-1} denotes the set of active nodes at $t-1$, and incoming influence weights are normalized such that $\sum_{u^{(i)} \in N^i_{in}(v)} w_{u^{(i)},v^{(i)}} \leq 1$.
- Interlayer propagation. When a node $u^{(i)}$ in layer i becomes active at timestamp $t \geq 0$, it attempts to activate its corresponding node $u^{(j)}$ in each other layer $j \neq i$ at $t+1$ with probability $w_{u^{(i)},u^{(j)}}$.

Activated nodes remain active throughout the propagation, which terminates when no further nodes can be activated.

Example 1. *Figure 2 illustrates the influence propagation process under the hybrid propagation model. Let $v^{(\ell)}_i$ denote node v_i in layer ℓ. Starting with seed node $v^{(1)}_0$, the propagation unfolds as follows:*

- *$t = 0$: Seed node $v^{(1)}_0$ is initially active.*
- *$t = 1$: Nodes $v^{(2)}_0$ and $v^{(1)}_2$ are activated by $v^{(1)}_0$ with influences 0.2 and 0.5 via interlayer and intralayer edges, respectively.*
- *$t = 2$: In layer 1, node $v^{(1)}_3$ is activated by $v^{(1)}_2$. In layer 2, although $v^{(2)}_2$ receives influence from $v^{(2)}_0$, it cannot be activated since $w_{v^{(2)}_0,v^{(2)}_2} = 0.1 < \theta^{(2)}_{v_2} = 0.6$. However, $v^{(2)}_2$ is successfully activated by interlayer influence from $v^{(1)}_2$.*
- *$t = 3$: Nodes $v^{(2)}_3$ and $v^{(2)}_4$ are activated by $v^{(2)}_2$.*
- *$t = 4$: Node $v^{(1)}_4$ is activated by $v^{(2)}_4$ via interlayer edge. No further activations are possible, and the process ends with active nodes $\{v^{(1)}_0, v^{(1)}_2, v^{(1)}_3, v^{(1)}_4\}$ in layer 1 and $\{v^{(2)}_0, v^{(2)}_2, v^{(2)}_3, v^{(2)}_4\}$ in layer 2.*

3.2 Problem Definition

Based on the hybrid propagation model, we present the formal problem definition of MLIM. Let S be a set of seed nodes. For each layer $i \in L$, let $I(G_i, S)$ denote the set of nodes in layer i eventually activated under the hybrid propagation model, including nodes activated via both intralayer and interlayer propagation. The expected

influence spread of S on layer i is defined as:

$$\sigma(S, G_i) = \mathbb{E}[|I(G_i, S)|]. \quad (1)$$

Definition 1. (Expected influence spread). *Given the expected influence spread $\sigma(S, G_i)$ of seed set S on each layer $i \in L$, the expected influence spread of S on the multilayer graph G is defined as:*

$$\sigma(S, G) = \sum\nolimits_{i \in L} \sigma(S, G_i). \quad (2)$$

This formulation sums the expected influence across all layers to capture the *total cross-platform exposure*. It aligns with practical marketing objectives where repeated user activations across multiple platforms amplify overall campaign effectiveness [7]. Notably, our framework can be extended to alternative influence metrics, such as $\sigma(S, G) = \mathbb{E}[|\cup_{i \in L} I(G_i, S)|$. A detailed discussion of this extension is provided in Appendix B. We now formally define the Mlim problem as follows.

Problem 1. (Mlim). *Given a multilayer graph $G(V, E, L)$ and a budget k, the objective of Mlim is to find a seed node set $S \subset V$ of size k that maximizes the total expected influence spread. Formally:*

$$S := \arg\max_{S \subset V, |S| = k} \sigma(S, G). \quad (3)$$

We establish the following properties of the Mlim problem under the hybrid propagation model.

Theorem 1. *The Mlim problem is* **NP***-hard under the hybrid propagation model.*

Theorem 2. *Given a multilayer graph $G(V, E, L)$ and a seed node set $S \subset V$, it is* #**P***-hard to compute the exact value of $\sigma(S, G)$ under the hybrid propagation model.*

Theorem 3. *Given a multilayer graph $G(V, E, L)$, the expected influence spread function $\sigma(S, G)$ is monotone and submodular w.r.t. S under the hybrid propagation model.*

4 Approximation Algorithms

In this section, we propose Mlim-Greedy and its scalable version STARIM with $(1 - 1/e - \epsilon)$ approximation for the Mlim problem.

4.1 Mlim-Greedy Algorithm

Since Mlim is monotone and submodular under the hybrid propagation model, we propose Mlim-Greedy, which extends the classical greedy framework [28] by iteratively selecting nodes that maximize influence spread across all layers. Starting with an empty seed set S, it iteratively computes the marginal gain $\Delta(v|S) = \sum_{i \in L}(\sigma(S \cup \{v\}, G_i) - \sigma(S, G_i))$ for all $v \in V \setminus S$, selects the node v^* with maximum gain, and adds it to S until $|S| = k$. Detailed pseudo-code is provided in Algorithm 3 in Appendix A.1. The approximation guarantee of Mlim-Greedy is established in Lemma 1.

Lemma 1. *Let S^* denote the optimal solution to the Mlim problem. The solution S returned by Mlim-Greedy satisfies:*

$$\sigma(S, G) \geq (1 - 1/e) \cdot \sigma(S^*, G). \quad (4)$$

4.2 STARIM Algorithm

Mlim-Greedy involves numerous computations of influence spread to determine the seed set. As this computation is #**P**-hard (Theorem 2), approximation methods are necessary. Existing methods like Monte Carlo (MC) [28] and Reverse Influence Sampling (RIS) [6] have been widely adopted for single-layer graphs. While RIS significantly improves efficiency over MC, it cannot be directly extended to multilayer graphs. The key issue is that existing RIS methods sample nodes uniformly from the entire graph under a single propagation model, whereas in multilayer graphs, different layers contain different numbers of nodes and follow distinct propagation patterns. A naive extension of RIS to multilayer graphs introduces bias in two aspects: layers with different sizes are disproportionately sampled, and distinct propagation models across layers are not properly considered. Therefore, we extend the RIS with a layer-weighted influence sampling strategy for multilayer graphs.

Multilayer Reverse Influence Sampling. We propose Multilayer Reverse Influence Sampling (MRIS) technique to effectively estimate $\sigma(S, G)$ on multilayer graphs through a two-stage sampling approach: first selecting a layer proportionally to its size, then uniformly selecting a node in that layer for reverse propagation under the layer-specific propagation dynamic. Specifically, we introduce the concept of a *Multilayer Reverse Reachable* (M-RR) set under the hybrid propagation model, which is generated through three steps:

(1) Select a layer $i \in L$ with probability $p = |V_i|/|V|$;
(2) Select a node $v \in V_i$ uniformly at random from layer i;
(3) Perform reverse propagation from v to construct a sample set R containing all nodes that can activate v (including v).

The reverse propagation proceeds as follows: (i) For intralayer edges under the IC model, we conduct reverse stochastic BFS. For each encountered node $u \in V_i$, examine its in-neighbors $a \in N^i_{in}(u)$ in layer i. Each edge (a, u) is traversed with probability $w_{a,u}$ if a is unvisited, and ignored with $1 - w_{a,u}$; (ii) For intralayer edges under the LT model, we conduct reverse random walk. For each encountered node u in layer i, stop at u with probability $1 - \sum_{a \in N^i_{in}(u)} w_{a,u}$, or continue to an in-neighbor $b \in N^i_{in}(u)$ with $\sum_{a \in N^i_{in}(v)} w_{a,u}$; (iii) For interlayer edges, when encountering a node $u \in V_i$ in layer $i \in L$, examine its interlayer in-neighbors $u' \in V_j$ $(j \neq i)$ and traverse edge (u', u) with probability $w_{u',u}$. The three mechanisms are handled simultaneously and independently.

The M-RR set R consists of all nodes visited during this process. Both the semantic definition (all nodes that can activate v) and the operational definition (all nodes visited during reverse propagation) are equivalent, established via the following coupling argument. We pre-sample a random graph G': for each IC intralayer edge (a, u), retain it with probability $w_{a,u}$; for each LT intralayer node v, select one in-edge (b, v) as a live edge with probability proportional to $w_{b,v}$; for each interlayer edge (u', u), retain it with probability $w_{u',u}$. Under G', a node u can activate v iff there exists a directed path from u to v in G', which is exactly the set of nodes visited during reverse propagation from v, establishing their equivalence.

Based on the generated M-RR sets, we design an unbiased estimator for influence spread over the multilayer graph. Given a seed set S and a random M-RR set R with *sampled source node v on layer $i \in L$*, define the random variable $\Lambda^i_R(S, G) = 1$ iff S intersects R, i.e., $S \cap R \neq \emptyset$, and $\Lambda^i_R(S, G) = 0$ otherwise. This indicates whether S can activate node v. Given a collection $\mathcal{R}$ of random M-RR sets, we define $f^{\mathcal{R}}(S, G_i)$ and $f^{\mathcal{R}}(S, G)$ as unbiased estimators of $\sigma(S, G_i)$ for layer i and $\sigma(S, G)$ for the entire multilayer graph. Formally,

$$f^{\mathcal{R}}(S, G) = \sum\nolimits_{i \in L} f^{\mathcal{R}}(S, G_i) = \sum\nolimits_{i \in L} \frac{|V_i|}{|\mathcal{R}_i|} \cdot \sum\nolimits_{R \in \mathcal{R}_i} \Lambda^i_R(S, G), \quad (5)$$

Algorithm 1 STARIM

Input: $G(V, E, L)$, k, ϵ, δ
Output: S

$S \leftarrow \emptyset$; $\theta_1 \leftarrow \max\{|V_1|, |V_2|, \ldots, |V_{|L|}|\}$; $\tau \leftarrow 1$;
while true do
  Generate $\mathcal{R}_1$ and $\mathcal{R}_2$ with $|\mathcal{R}_1| = |\mathcal{R}_2| = \theta_\tau$;
  $S \leftarrow$ NodeSelection$(\mathcal{R}_1, k)$;
  Compute $f^{\mathcal{R}_1}(S, G)$ and $f^{\mathcal{R}_2}(S, G)$ by Eq.(5);
  $\mu \leftarrow f^{\mathcal{R}_1}(S, G)/f^{\mathcal{R}_2}(S, G)$;
  $\epsilon_1 \leftarrow (\epsilon_1+1)(\epsilon_1+2)/\epsilon_1^2 = f^{\mathcal{R}_2}(S, G)/\ln(5\tau^2/\delta) \cdot \min_{\mathcal{R}_i \subseteq \mathcal{R}_2}\{\frac{|\mathcal{R}_i|}{|V_i|}\}$;
  $\epsilon_2 \leftarrow 2\epsilon_1 + 2/\epsilon_2^2 = f^{\mathcal{R}_2}(S, G)/\ln(5\tau^2/\delta) \cdot \min_{\mathcal{R}_i \subseteq \mathcal{R}_1}\{\frac{|\mathcal{R}_i|}{|V_i|}\}$;
  if $0 < \mu \leq \frac{1-1/e}{1-1/e-\epsilon} \cdot \frac{1-\epsilon_2}{1+\epsilon_1}$, ϵ_1, $\epsilon_2 \in (0, 1)$ **then break**;
  if $\min_{\mathcal{R}_i \in \mathcal{R}_1}\{\frac{|\mathcal{R}_i|}{|V_i|}\} \geq (8+2\epsilon)(1+\epsilon_1)\frac{\ln\frac{6}{\delta}+|V|\ln 2}{\epsilon^2 \cdot f^{\mathcal{R}_2}(S,G)}$ **then break**;
  $\tau \leftarrow \tau + 1$; $\theta_\tau \leftarrow 2\theta_\tau$;
Return S.

where $\mathcal{R}_i \subseteq \mathcal{R}$ denotes the set of M-RR sets whose sampled source nodes are on layer i. To verify unbiasedness, it suffices to show $\mathbb{E}[f^{\mathcal{R}}(S, G_i)] = \sigma(S, G_i)$ and $\mathbb{E}[f^{\mathcal{R}}(S, G)] = \sigma(S, G)$, where $f^{\mathcal{R}}(S, G_i) = \frac{|V_i|}{|\mathcal{R}_i|} \cdot \sum_{R \in \mathcal{R}_i} \Lambda_R^i(S, G)$ and $f^{\mathcal{R}}(S, G) = \sum_{i \in L} f^{\mathcal{R}}(S, G_i)$. Since each node $v \in V_i$ is sampled uniformly with probability $1/|V_i|$, then, $\mathbb{E}\left[\frac{|V_i|}{|\mathcal{R}_i|} \cdot \sum_{R \in \mathcal{R}_i} \Lambda_R^i(S, G)\right] = |V_i| \cdot \frac{1}{|V_i|} \cdot \sum_{v \in V_i} \Pr[v \in I(G_i, S)] = \sigma(S, G_i)$. Linearity of expectation, $\mathbb{E}\left[f^{\mathcal{R}}(S, G)\right] = \sum_{i \in L} \mathbb{E}\left[f^{\mathcal{R}}(S, G_i)\right] = \sum_{i \in L} \sigma(S, G_i) = \sigma(S, G)$.

STARIM. Based on the discussions above, we propose the *Scalable mulTilAyeR Influence Maximization (STARIM)* algorithm. STARIM uses MRIS to estimate influence spread and iteratively selects nodes with maximum marginal gain. The returned S has a theoretical guarantee with sufficient M-RR sets. A key challenge is determining the number of M-RR sets $|\mathcal{R}|$ needed to ensure approximation guarantees without excessive sampling. STARIM addresses this through an adaptive trial-and-error strategy [54]: it generates two independent M-RR sets $\mathcal{R}_1$ and $\mathcal{R}_2$, and during M-RR set generation, the sample size of both $\mathcal{R}_1$ and $\mathcal{R}_2$ are doubled incrementally until the algorithm terminates when either (i) the approximation ratio $\mu = f^{\mathcal{R}_1}(S, G)/f^{\mathcal{R}_2}(S, G)$ between two independently estimated influence spreads satisfies $0 < \mu \leq (\frac{1-1/e}{1-1/e-\epsilon} \cdot \frac{1-\epsilon_2}{1+\epsilon_1})$, or (ii) the minimum ratio of sampled M-RR sets per layer meets $\min_{R_i \in \mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\} \geq (8+2\epsilon)(1+\epsilon_1)\frac{\ln\frac{6}{\delta}+|V|\ln 2}{\epsilon^2 \cdot f^{\mathcal{R}_2}(S,G)}$. Algorithm 1 presents STARIM, and Algorithm 2 presents NodeSelection for greedy seed selection using $\mathcal{R}_1$. *The detailed theoretical derivations for these two conditions are provided in Appendix C.5 and Appendix C.6 in [10].*

Lemma 2 establishes concentration bounds for the estimators $f^{\mathcal{R}_1}(S^*, G)$ and $f^{\mathcal{R}_2}(S, G)$ in each round of STARIM, and Theorem 4 provides the approximation guarantee of STARIM. Let S^* denote the optimal solution to the MLIM problem, and $\delta, \epsilon \in (0, 1)$ are tunable error parameters.

Lemma 2. *With probability at least $1 - \frac{2\delta}{3}$, for each iteration of Algorithm 1, where $\epsilon_1, \epsilon_2 > 0$, we have*

$$f^{\mathcal{R}_2}(S, G) \leq (1+\epsilon_1) \cdot \sigma(S, G), \tag{6}$$

$$f^{\mathcal{R}_1}(S^*, G) \geq (1-\epsilon_2) \cdot \sigma(S^*, G). \tag{7}$$

Theorem 4. (Theoretical guarantee). *With a probability of at least $1 - \delta$ for any δ, the solution S returned by STARIM satisfies:*

$$\sigma(S, G) \geq (1 - 1/e - \epsilon) \cdot \sigma(S^*, G). \tag{8}$$

Algorithm 2 NodeSelection

Input: $\mathcal{R}_1$, k
Output: S

$S \leftarrow \emptyset$;
For each layer $i \in L$, let $\mathcal{R}_1^i \subseteq \mathcal{R}_1$ be the set of M-RR sets with sampled source nodes on layer i;
while $|S| < k$ **do**
  for each $v \in V \setminus S$ **do**
    $\Delta_{\mathcal{R}_1}(v) \leftarrow \sum_{i \in L} \frac{|V_i|}{|\mathcal{R}_1^i|} \times |\{R \in \mathcal{R}_1^i : v \in R\}|$;
  $v^* \leftarrow \arg\max_{v \in V \setminus S} \Delta_{\mathcal{R}_1}(v)$, $S \leftarrow S \cup \{v^*\}$;
  Remove from $\mathcal{R}_1$ all M-RR sets covered by v^*;
Return S.

Theorem 5. (Time complexity). *The expected time complexity of Algorithm 1 is $O(\frac{(\ln 1/\delta + \ln|V|) \cdot |E| \cdot \sigma(\{v^*\})}{\epsilon^2})$, where v^* is the node in G with the largest expected spread.*

5 Learning-based Framework

In this section, we propose LGQIM, a two-stage learning framework that combines multilayer node influence prediction with multilayer adaptive seed selection.

5.1 Framework Overview

While STARIM effectively addresses the MLIM problem with theoretical guarantees, its computational cost can be prohibitive for large-scale multilayer graphs in real-time applications, due to the repeated graph traversals for M-RR set generation. To address this limitation, we propose LGQIM, a learning-based framework that improves efficiency by avoiding online M-RR set sampling. Figure 3 illustrates the training workflow of LGQIM, which consists of three main components: **(1) Training Data Generation.** We first apply the *MRIS* method to compute the influence score of each node, i.e., the influence spread when each node is selected as the sole seed. These scores serve as labeled data for supervised training of node influence prediction process. **(2) Multilayer Node Influence Prediction.** Directly computing node influence scores at test time would require expensive online M-RR set sampling. Therefore, we employ graph embedding learning to efficiently predict influence scores through supervised learning. The learned embeddings capture both intralayer and interlayer structural features that encode propagation dynamics across the multilayer graph. The predicted scores are then combined with structural features as input representations for the subsequent reinforcement learning phase. **(3) Multilayer Adaptive Seed Selection.** Since selecting the optimal seed set requires addressing the combinatorial nature of the problem, we formulate this as a sequential decision-making task and solve it through deep reinforcement learning. The seed selection process is modeled as a Markov Decision Process, where the state represents the current seed set, an action corresponds to selecting a new node, and the reward reflects the marginal influence gain. This approach learns an optimal policy that sequentially constructs the final seed set by balancing exploration and exploitation.

5.2 Multilayer Node Influence Prediction

We train a Graph Convolutional Network (GCN) [21] model in a supervised manner to predict the influence score of each node in

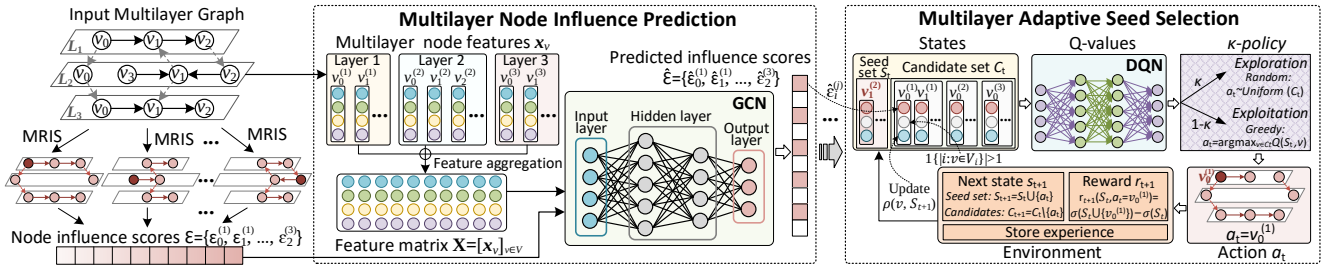


Figure 3: Workflow of the training phase of LGQIM

a given multilayer graph G. The objective is to learn each node embeddings that capture both intralayer and interlayer structural features, enabling accurate influence prediction directly from the multilayer graph structure without M-RR set sampling. To generate training data, we first apply the *MRIS* method on G to compute the influence score of each node $v \in V$, i.e., the influence spread when v is selected as the sole seed. This yields layer-wise influence scores $\mathcal{E} = \{\mathcal{E}^{(1)}, \mathcal{E}^{(2)}, \ldots, \mathcal{E}^{(|L|)}\}$ where $\mathcal{E}^{(\ell)} = \{\epsilon_v^{(\ell)} : v \in V_\ell\}$ with $\epsilon_v^{(\ell)} \in \mathbb{R}^+$, and V_ℓ is the set of nodes in layer $\ell \in L$. The graph structure G together with the scores $\mathcal{E}$ are then used to train the GCN and learn its parameters Θ_S. Once trained, the model predicts influence scores for nodes in a given graph. In what follows, we present the key components of the GCN training process.

Node feature construction. We embed each node into a feature vector that captures its structural properties in the multilayer graph. Specifically, for every node $v \in V$, the initial representation $\boldsymbol{x}_v$ is defined as a 4-dimensional vector:

$$\boldsymbol{x}_v = \Big[|N_{out}(v)|,\ |N_{in}(v)|,\ \textstyle\sum_{u \in N^{\ell}_{out}(v)} w_{v,u},\ \sum_{u \in N_{out}(v) \setminus N^{\ell}_{out}(v)} w_{v,u}\Big], \quad (9)$$

where v is a node in layer ℓ. The four features in $\boldsymbol{x}_v$ are interpreted as follows: $|N_{out}(v)|$ and $|N_{in}(v)|$ represent the out-degree and in-degree of v in the entire graph, $\sum_{u \in N^{\ell}_{out}(v)} w_{v,u}$ is the total outgoing edge weight within layer ℓ, and $\sum_{u \in N_{out}(v) \setminus N^{\ell}_{out}(v)} w_{v,u}$ is the total outgoing edge weight to other layers from v. These features collectively encode each node's connectivity and propagation capability both within and across layers. We denote the feature matrix for all nodes in V as:

$$\boldsymbol{X} = [\boldsymbol{x}_v]_{v \in V} \in \mathbb{R}^{|V| \times 4}, \quad (10)$$

where each row corresponds to the 4-dimensional feature vector of a node. This feature matrix $\boldsymbol{X}$ serves as the input to the GCN.

GCN architecture. The GCN refines node representations by iteratively aggregating information from their neighbors. At the $(l+1)$-th layer, the node embeddings are updated as:

$$H^{(l+1)} = \phi\Big(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\Big), \quad (11)$$

where $\tilde{A}$ is the adjacency matrix across all layers with self-loops, $\tilde{D}$ is the corresponding degree matrix, $W^{(l)}$ is the learnable weight matrix for GCN layer l, and ϕ is a non-linear activation function. The input features $H^{(0)} = \boldsymbol{X}$ encode node and layer attributes.

Supervised GCN training. Given the input node feature matrix $\boldsymbol{X}$ and the GCN architecture described above, the objective is to train the network to predict the influence score of each node in the graph. The ground-truth and predicted influence scores are denoted by $\mathcal{E} = \{\{\epsilon_v^{(\ell)} : v \in V_\ell\} : \ell \in L\}$ and $\hat{\mathcal{E}} = \{\{\hat{\epsilon}_v^{(\ell)} : v \in V_\ell\} : \ell \in L\}$, respectively. The model parameters Θ_S are learned by minimizing the mean squared error (MSE) between the predicted and true scores, which is formally defined as:

$$\mathcal{L}(\Theta_S) = \frac{1}{|V|} \sum\nolimits_{\ell \in L} \sum\nolimits_{v \in V_\ell} (\hat{\epsilon}_v^{(\ell)} - \epsilon_v^{(\ell)})^2. \quad (12)$$

The detailed pseudo-code for the supervised training framework is provided in Algorithm 4 in Appendix A.2.

5.3 Multilayer Adaptive Seed Selection

After the GCN has predicted the individual influence scores of nodes, we reformulate the MLIM problem as a sequential decision-making task and employ a Q-learning network [53]. The objective is to learn an optimal policy (i.e., a sequence of actions) for selecting seed nodes. The Q-learning network provides an end-to-end framework that connects the agent's perception to its actions, enabling it to automatically extract task-relevant features. In our setting, the agent is the machine that learns to select seed nodes optimally within an uncertain environment. The entire decision-making process is modeled as a Markov Decision Process: at each time step t, the agent observes the state s_t (i.e., the current set of selected nodes), performs an action a_t (i.e., selecting a new node), gets a reward r_{t+1}, and moves to the next state s_{t+1}. This interaction continues until an episode ends. Specifically, for a given set of selected nodes S and a candidate node $v \in V \setminus S$, we estimate the n-step reward $Q_n(S, v)$ of adding v to S using the surrogate function $Q'_n(S, v; \Theta_Q)$.

We formulate the Q-learning task in terms of state space, action, reward, and policy where the input consists of the set of nodes along with their predicted influence scores and structure features.

State space. At time step t, state s_t represents the current configuration, where some nodes have been selected into the seed set and others remain unselected. The process terminates at the final state s_k, where exactly k nodes have been selected. The state space thus characterizes the state of the system at any time step t in terms of the partially constructed solution S_t and the corresponding set of candidate nodes $C_t = V \setminus S_t$. For each candidate node $v \in C_t$, we define its feature representation as a 3-dimensional vector:

$$\boldsymbol{\lambda}_v = [\hat{\epsilon}_v^{(\ell)}, \rho(v, S_t), \mathbf{1}\{|\{i : v \in V_i\}| > 1\}]. \quad (13)$$

Here, $\hat{\epsilon}_v^{(\ell)}$ denotes the predicted influence score of node $v \in V_\ell$ where $\ell \in L$, reflecting its influence quality across the entire graph. Following [40], the term $\rho(v, S_t) = |N_{out}(v) \setminus \cup_{u \in S_t} N_{out}(u)|$ measures the number of out-neighbors of v not yet influenced by the current seed set, thereby capturing v's marginal contribution to influence propagation. The indicator function $\mathbf{1}\{|i : v \in V_i| > 1\}$ specifies whether v is an overlap node, i.e., one that belongs to

multiple layers (1 if true, 0 otherwise), where V_i denotes the node set of layer i. The representation of the candidate set C_t is then obtained by applying MaxPool over $\{\boldsymbol{\lambda}_v \mid v \in C_t\}$, denoted by $\boldsymbol{\lambda}_{C_t}$. $\boldsymbol{\lambda}_{S_t}$ is defined analogously as well. We adopt MaxPool as the aggregation function since it highlights the most promising candidate, effectively guiding the agent toward high-quality actions.

Action and reward. At each time step t, the action a_t corresponds to selecting a node $v \in C_t$ and adding it to the current seed set S_t. Once the action is taken, the environment transitions to a new state s_{t+1} and returns a reward $r_{t+1} \in \mathbb{R}$. The reward function is defined as the marginal influence gain obtained by adding v, namely,

$$r_{t+1}(S_t, a_t = v) = \sigma(S_{t+1}, G) - \sigma(S_t, G) = \sigma(S_t \cup \{v\}, G) - \sigma(S_t, G). \quad (14)$$

Optimal policy. The objective of Q-learning is to derive the optimal policy $\pi^* = (a_1, a_2, \ldots, a_k)$ maximizing the expected cumulative reward $\mathbb{E}[\sum_{i=1}^{k} r(S_i, a_i)]$, which corresponds to identifying the k seed nodes that yield the largest influence spread. The trained Q-learning network implicitly encodes this policy by estimating the n-step reward for each candidate action. Formally, the policy selects the node with the highest predicted value:

$$\pi^*(v|S_t) = \arg\max_{v \in C_t} Q'_n(S_t, v; \Theta_Q). \quad (15)$$

Q-learning network training. We adopt the Deep Q-Network (DQN) framework [42, 43] to learn the Q-function $Q(S, v; \Theta_Q)$, which is implemented as a deep neural network over graph embeddings. To enhance stability and efficiency, we employ experience replay: past transitions are stored and randomly sampled for off-policy updates, thereby mitigating correlations between consecutive experiences. Q-learning updates parameters in a single episode via Adam optimizer [29] to minimize the squared loss, formally:

$$\mathcal{L}(\Theta_Q) = \mathbb{E}\Big[\big(\gamma \cdot \max_{v \in V} Q'_n(S_{t+n}, v; \Theta_Q) + \sum_{i=0}^{n-1} r(S_{t+i}, v_{t+i}) - Q'_n(S_t, v; \Theta_Q)\big)^2\Big], \quad (16)$$

where γ is the decaying factor, which balances the importance of immediate reward with the predicted n-step future reward. Detailed pseudo-code is provided in Algorithm 5 in Appendix A.3.

5.4 Test Phase

Given a multilayer graph G and a budget k, the test phase proceeds in two stages. First, we perform a single forward pass through the trained GCN to predict the influence scores for all nodes. These scores are then used to construct node embeddings, which are then fed into the trained Q-learning network. The Q-network sequentially selects nodes according to the learned policy until k nodes have been selected, yielding the final seed node set.

6 Experiments

In this section, we empirically evaluate our proposed algorithms. All methods are executed on a Linux server with Intel(R) Xeon(R) 3.70 GHz CPU and 128GB of RAM. The source code is available at https://github.com/ZJU-DAILY/MLIM.

6.1 Experimental Settings

Dataset. We evaluate the proposed algorithms on nine real multilayer networks, as summarized in Table 1, where Citeseer and DBLP are collaboration networks, StackOverflow is temporal network and the others are social networks. For each undirected network, we treat every edge as bidirectional. Moreover, we adopt the *weighted-cascade* model [28] to assign influence weights, where the weight of edge (u, v) is defined as $w_{u,v} = 1/|N_{in}(v)|$. Interlayer edges connecting corresponding nodes across layers are weighted similarly.

Table 1: Dataset Statistics

Dataset	$\|V\|$	$\|E\|$	$\|L\|$	Type
Venetie (VT) [4]	1,380	19,941	43	Undirected
FF-TW-YT (FTY) [14]	11,863	87,396	3	Directed
Citeseer (CS) [39]	15,533	122,903	3	Undirected
DBLP [39]	41,892	661,883	2	Undirected
Twitter (TW) [4]	47,807	657,456	3	Undirected
MoscowAth (MA) [48]	133,364	309,952	3	Directed
Friendfeed (FF) [14]	1,531,015	20,204,535	3	Undirected
Obama (OB) [48]	3,452,114	6,711,448	3	Directed
StackOverflow (SF) [32]	2,601,977	63,497,050	9	Directed

Algorithms. We evaluate our proposed algorithms against representative baselines from three categories: approximation, heuristic, and learning-based approaches. Our methods include STARIM, a theoretical-based algorithm with approximation guarantees, and LGQIM, a learning-based framework combining GCN and Q-learning network. Specifically, the baselines include: KSN [30], an approximation algorithm applying IMM [56] on each layer with knapsack optimization; CBIM [50], a heuristic method based on community detection and edge weight ranking; and DeepIM [27], a deep learning approach selecting seed nodes based on learned structural features. We also include HighDegree (HD), which selects seeds with highest average out-degrees across layers, and Random (RAN), which uniformly selects seeds at random, as baselines.

Parameters. We evaluate algorithm performance under different parameter settings. In each experiment, we vary only one parameter while keeping the others at their default values. Experiments exceeding 24 hours are set as INF. For STARIM, we test the budget k and sampling error factor ϵ in the *MRIS* technique. The default values are set as follows: $k = 100$ for VT, FTY, and CS datasets, $k = 500$ for the others; $\epsilon = 0.2$ and $\delta = 0.01$ for all datasets. In all experiments, the influence spread of each algorithm is estimated using $2^5 \times 10^5$ M-RR sets generated independently of the evaluated algorithms. We repeat each algorithm 5 times and report the average value. For LGQIM, following [40], the GCN is trained for 500 epochs on SF and 1000 epochs on the others, with convolution depth 2 and embedding dimension 60. For n-step Q-learning network, we set $n = 2$, decaying factor $\gamma = 0.8$, hidden dimension 10, and learning rate 0.001. Exploration probability κ anneals linearly from 1.0 to 0.1. Batch size is 20 for FF, OB, and SF, and 200 for the others. Each dataset is split evenly for training and testing. In all experiments, the sampled subgraph comprises 10% of the nodes from the entire graph, following [33, 40].

6.2 Experiment Results

Exp-1: Overall performance. We evaluate the effectiveness and efficiency of all methods on all datasets under the default settings. Figure 4 shows the influence spread and running time. The results indicate that STARIM and LGQIM consistently achieve the highest influence spread, surpassing all the other algorithms. This is because STARIM employs our greedy seed selection strategy (Algorithm 2)

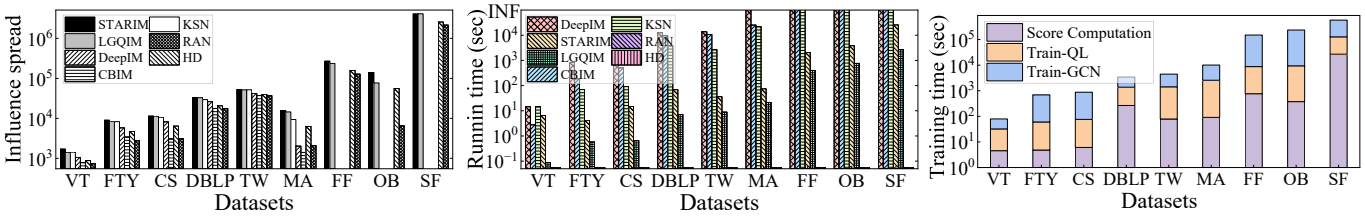


Figure 4: Overall performance on all datasets

Figure 5: Training time of LGQIM

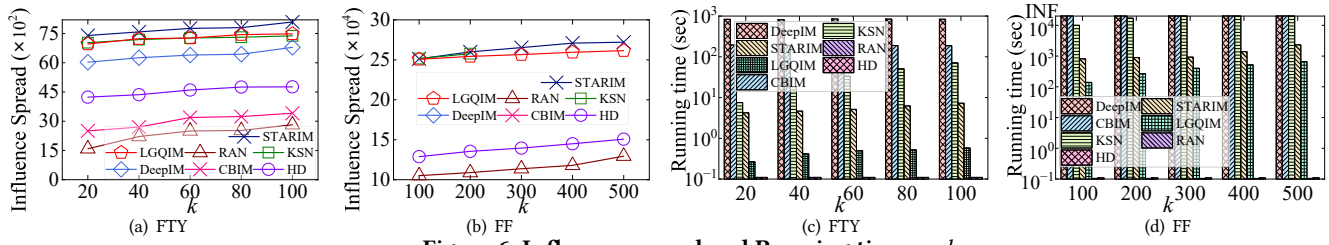


Figure 6: Influence spread and Running time vs. k

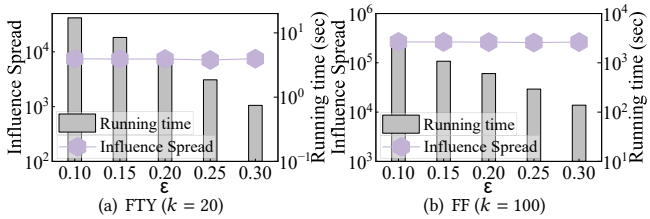


Figure 7: Influence spread and Running time vs. ϵ

with theoretical guarantees, while LGQIM outperforms the other learning-based solution DeepIM in influence spread. Note that on large datasets (FF, OB, and SF), DeepIM, CBIM, and KSN fail to complete within a reasonable time. Regarding efficiency, STARIM is consistently faster than KSN, primarily due to early termination of M-RR set generation. In contrast, KSN applies IMM independently on each layer and performs knapsack-based optimization, requiring enumeration of numerous seed set combinations. In addition, LGQIM achieves significantly lower running time than STARIM, highlighting the efficiency advantage of our learning-based framework. While heuristic algorithms RAN and HD run faster due to lack of theoretical guarantees, their influence spreads are generally lower than those of STARIM and LGQIM.

Exp-2: Varying k. We examine the impact of seed size k on the FTY and FF datasets. Other datasets exhibit similar trends and are omitted due to space constraints. As shown in Figures 6(a) and 6(b), the influence spread of all methods grows with larger k, since a larger seed set can activate more nodes. STARIM, LGQIM, and KSN consistently achieve the highest influence spread. Figures 6(c) and 6(d) show that STARIM, LGQIM, DeepIM and KSN all exhibit longer running time as k increases. Among them, LGQIM achieves the best efficiency by avoiding costly M-RR set generation. Moreover, STARIM and LGQIM achieve up to two orders of magnitude speedup over KSN, benefiting from early termination in M-RR set generation and elimination of online M-RR set sampling.

Exp-3: Varying ϵ. We investigate the effect of ϵ, the sampling error factor in the approximation achieved of the *MRIS*. Since STARIM is the only method that leverages *MRIS* for seed selection and provides theoretical guarantee, we evaluate its influence spread and running time under different ϵ values. Due to the consistent results, we reported results only on FTY and FF. Figure 7 shows that influence spread remains relatively stable as ϵ varies, since the approximation indicates the worst-case performance while empirical performance is generally robust. Moreover, running time decreases as ϵ grows due to early termination in STARIM (Line 9 in Algorithm 1), which reduces the number of M-RR sets generated. According to Theorem 5, M-RR set generation dominates the computational cost of STARIM, leading to overall reduction in running time.

Table 2: Ablation study

Dataset	Influence spread			Running time (sec)		
	S+Q	G+Q	LGQIM-	S+Q	G+Q	LGQIM-
VT (k = 100)	1227.3	1217.1	1030.4	4.6362	0.089	0.076
FTY (k = 100)	8089.7	7479.7	6168.5	5.0874	0.5753	0.5564
CS (k = 100)	9128.4	9338.8	7020.6	6.4352	0.6608	0.7195
DBLP (k = 500)	26588.5	26929.2	22366.3	271.53	7.1983	8.7275
TW (k = 500)	42299.3	42727.4	35941.5	86.442	8.8831	10.1788
MA (k = 500)	12614.7	13519.9	10554.7	30.080	21.713	30.546
FF (k = 500)	240661	239503	233849	1416.1	704.80	1283.2
OB (k = 500)	55291.8	56695.7	36326.0	1064.0	777.17	820.67
SF (k = 500)	4139860	4116730	4069729	29425	2698.1	3047.3

* **S+Q**: **S**core Computation + Q-learning network; **G+Q**: **G**CN + Q-learning network.

Exp-4: Training time analysis. We analyze the training overhead of LGQIM. Figure 5 shows the time distribution across training phases. Score Computation applies the *MRIS* technique to obtain influence scores for all nodes as GCN labels. Train-GCN and Train-QL denote the training processes of the GCN and Q-learning network, respectively. Among these phases, Train-GCN takes the longest time due to iterative message passing over the entire graph, where each node aggregates information from its neighbors. In contrast, Train-QL is much faster as it directly uses GCN-predicted scores and performs feed-forward updates in a relatively small state space.

Exp-5: Ablation study. We evaluate key components of LGQIM through an ablation study in Table 2. We first evaluate the GCN component by comparing *MRIS*-computed versus GCN-predicted influence scores as input to the Q-learning network. While both achieve nearly identical influence spread, GCN-predicted scores reduce running time by an order of magnitude on average, confirming that GCN serves as an effective lightweight surrogate for *MRIS*. Additionally, we compare LGQIM-, which excludes multi-layer information from the framework. Results show that LGQIM- achieves lower influence spread with comparable running time,

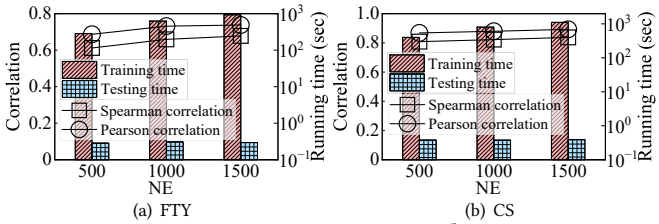


Figure 8: Hyperparameter analysis vs. *NE*

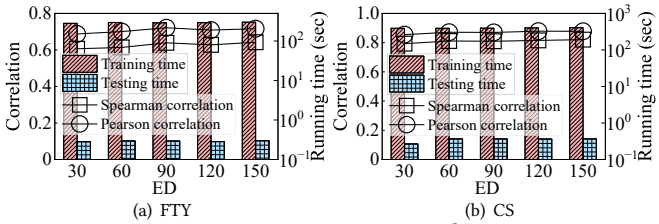


Figure 9: Hyperparameter analysis vs. *ED*

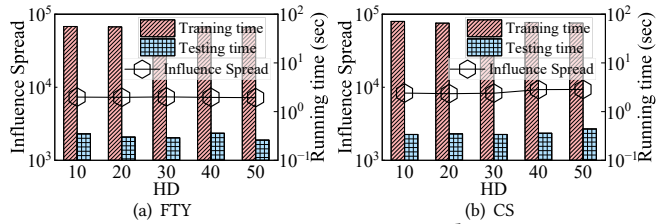


Figure 10: Hyperparameter analysis vs. *HD*

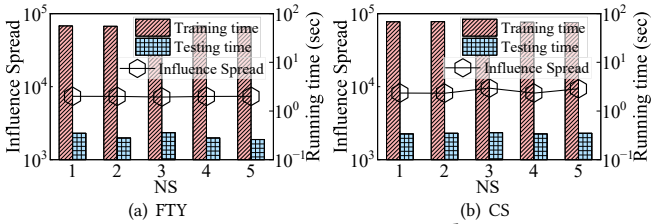


Figure 11: Hyperparameter analysis vs. *NS*

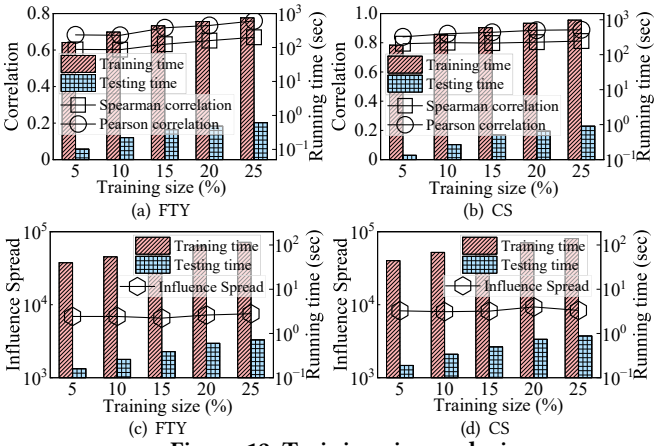


Figure 12: Training size analysis

demonstrating that multilayer-aware architectures are essential for both accurate influence prediction and effective seed selection.

Exp-6: Hyperparameter analysis. We analyze the impact of key hyperparameters of LGQIM on the FTY and CS datasets by varying *Number of Epochs* (*NE*) in {500, 1000, 1500}, *Embedding Dimension* (*ED*) in {30, 60, 90, 120, 150}, *Hidden Dimension* (*HD*) in {10, 20, 30, 40, 50}, and *n_step* (*NS*) in {1, 2, 3, 4, 5}. **(1)** As shown in Figure 8, increasing NE in GCN leads to higher Spearman and Pearson correlations, as more epochs enable better parameter optimization and neighbor information aggregation, capturing higher-order structural patterns more effectively. Training time increases with larger NE, while testing time remains nearly constant since inference requires only a single forward pass. **(2)** In Figure 9, GCN performance improves as ED increases. Higher dimensional embeddings better represent node features and structural patterns, enabling more accurate influence score estimation. Training time increases slightly with larger ED due to additional parameters and heavier matrix operations, whereas testing time is almost unaffected. **(3)** In Figures 10 and 11, increasing HD and NS in the *Q*-learning network has little effect on training time, testing time, or influence spread. This is because the *Q*-learning component mainly performs discrete action selection with low structural complexity, making it less sensitive to these hyperparameters, indicating robustness across different settings.

Exp-7: Training size analysis. We evaluate the effect of training data size on GCN (Figures 12(a) and 12(b)) and *Q*-learning network (Figures 12(c) and 12(d)). Both training and testing sets are formed by selecting top-ranked nodes by influence scores. We observe that using only 5% of nodes achieves nearly identical performance to using 25% of nodes, demonstrating that LGQIM can effectively learn a generalized policy from relatively small training data. As expected, both training and testing time increase with dataset size.

Table 3: Cross-graph generalization

k	VT			FTY		
	STARIM	LGQIM	LGQIM-CS	STARIM	LGQIM	LGQIM-CS
20	1175.32	1166.66	993.114	7396.9	6960.85	5934.53
40	1182.43	1186.44	1012.63	7580.42	7233.73	6180.29
60	1215.32	1193.63	1013.922	7762.24	7266.86	6197.16
80	1223.38	1213.65	1017.56	7799.17	7441.82	6219.41
100	1227	1217.07	1038.845	8089.71	7479.77	6380.36

Exp-8: Cross-Graph Generalization. We evaluate the generalization ability of LGQIM across different multilayer graphs. Specifically, we train LGQIM on CS and directly apply it to VT and FTY. As shown in Table 3, LGQIM-CS achieves competitive influence spread compared to STARIM, and remains close to LGQIM trained on the target datasets, demonstrating that LGQIM generalizes reasonably well across different multilayer graphs.

7 Conclusions

In this paper, we investigate the Mlim problem, which seeks to select a small seed set to maximize influence spread in multilayer networks. We introduce a hybrid propagation model that captures multilayer diffusion patterns. We then develop Mlim-Greedy with $(1-1/e)$ approximation, STARIM with $(1-1/e-\epsilon)$ approximation based on *MRIS*, and LGQIM integrating multilayer representation learning with adaption seed selection. Experiments demonstrate that our algorithms are effective, efficient, and scalable.

Acknowledgments

This work was supported in part by the NSFC under Grants No. (U23A20296, 62025206, and 62302444), and Zhejiang Province's "Lingyan" R&D Project under Grant No. 2024C01259.

References

[1] Oumaima Achour and Lotfi Ben Romdhane. 2025. A theoretical review on multiplex influence maximization models: Theories, methods, challenges, and future directions. *Expert Systems with Applications* 266 (2025), 125990.

[2] Dawood Al Abri and Shahrokh Valaee. 2020. Diversified viral marketing: The power of sharing over multiple online social networks. *Knowledge-Based Systems* 193 (2020), 105430.

[3] Saleem Alhabash and Mengyan Ma. 2017. A tale of four platforms: Motivations and uses of Facebook, Twitter, Instagram, and Snapchat among college students? *Social media+ society* 3, 1 (2017), 2056305117691544.

[4] Jacopo A Baggio, Shauna B BurnSilver, Alex Arenas, James S Magdanz, Gary P Kofinas, and Manlio De Domenico. 2016. Multiplex social ecological network analysis reveals how social changes affect community robustness more than resource depletion. *Proceedings of the National Academy of Sciences* 113, 48 (2016), 13708–13713.

[5] Prithu Banerjee, Wei Chen, and Laks VS Lakshmanan. 2019. Maximizing welfare in social networks under a utility driven influence diffusion model. In *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*. 1078–1095.

[6] Christian Borgs, Michael Brautbar, Jennifer Chayes, and Brendan Lucier. 2014. Maximizing social influence in nearly optimal time. In *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 946–957.

[7] Builtin. 2023. How the Mere Exposure Effect Works in Marketing and Advertising. https://builtin.com/articles/mere-exposure-effect

[8] Xueqin Chang, Jiajie Fu, Qing Liu, Yunjun Gao, and Baihua Zheng. 2025. Time-aware influence minimization via blocking social networks. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 557–570.

[9] Xueqin Chang, Xiangyu Ke, Lu Chen, Congcong Ge, Ziheng Wei, and Yunjun Gao. 2023. Host profit maximization: Leveraging performance incentives and user flexibility. *Proceedings of the VLDB Endowment* 17, 1 (2023), 51–64.

[10] Xueqin Chang, Ruize Liu, Qing Liu, Baihua Zheng, and Yunjun Gao. 2026. Approximation and Learning-based Algorithms for Influence Maximization in Multilayer Social Networks. https://github.com/ZJU-DAILY/MLIM

[11] Wei Chen, Yifei Yuan, and Li Zhang. 2010. Scalable influence maximization in social networks under the linear threshold model. In *2010 IEEE International Conference on Data Mining (ICDM)*. 88–97.

[12] Shuang Cui, Kai Han, Jing Tang, and He Huang. 2023. Constrained subset selection from data streams for profit maximization. In *Proceedings of the ACM Web Conference (WWW)*. 1822–1831.

[13] DATAREPROTAL. 2025. Global social media Statistics. https://datareportal.com/social-media-users?

[14] Mark E. Dickison, Matteo Magnani, and Luca Rossi. 2016. *Multilayer Social Networks*. Cambridge University Press.

[15] Nguyen Hoang Khoi Do, Tanmoy Chowdhury, Chen Ling, Liang Zhao, and My T Thai. 2024. MIM-Reasoner: Learning with Theoretical Guarantees for Multiplex Influence Maximization. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 2296–2304.

[16] Pedro Domingos and Matt Richardson. 2001. Mining the network value of customers. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. 57–66.

[17] Yuting Feng, Vincent YF Tan, and Bogdan Cautis. 2024. Influence Maximization via Graph Neural Bandits. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 771–781.

[18] Qintian Guo, Chen Feng, Fangyuan Zhang, and Sibo Wang. 2023. Efficient algorithm for budgeted adaptive influence maximization: An incremental RR-set update approach. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 1–26.

[19] Qintian Guo, Sibo Wang, Zhewei Wei, Wenqing Lin, and Jing Tang. 2022. Influence Maximization Revisited: Efficient Sampling with Bound Tightened. *ACM Transactions on Database Systems (TODS)* (2022).

[20] Michael Haenlein, Ertan Anadol, Tyler Farnsworth, Harry Hugo, Jess Hunichen, and Diana Welte. 2020. Navigating the new era of influencer marketing: How to be successful on Instagram, TikTok, & Co. *California management review* 63, 1 (2020), 5–25.

[21] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*. 1025–1035.

[22] Farnoosh Hashemi, Ali Behrouz, and Laks VS Lakshmanan. 2022. Firmcore decomposition of multilayer networks. In *Proceedings of the ACM Web Conference (WWW)*. 1589–1600.

[23] Keke Huang, Ruize Gao, Bogdan Cautis, and Xiaokui Xiao. 2024. Scalable continuous-time diffusion framework for network inference and influence estimation. In *Proceedings of the ACM Web Conference (WWW)*. 2660–2671.

[24] Yiqian Huang, Shiqi Zhang, Laks VS Lakshmanan, Wenqing Lin, Xiaokui Xiao, and Bo Tang. 2024. Efficient and Effective Algorithms for A Family of Influence Maximization Problems with A Matroid Constraint. *Proceedings of the VLDB Endowment* 18, 2 (2024), 117–129.

[25] Tianyuan Jin, Yu Yang, Renchi Yang, Jieming Shi, Keke Huang, and Xiaokui Xiao. 2021. Unconstrained submodular maximization with modular costs: Tight approximation and application to profit maximization. *Proceedings of the VLDB Endowment* 14, 10 (2021), 1756–1768.

[26] Mahender Katukuri, Maheswari Jagarapu, et al. 2022. CIM: clique-based heuristic for finding influential nodes in multilayer networks. *Applied Intelligence* 52, 5 (2022), 5173–5184.

[27] Mohammad Mehdi Keikha, Maseud Rahgozar, Masoud Asadpour, and Mohammad Faghih Abdollahi. 2020. Influence maximization across heterogeneous interconnected networks based on deep learning. *Expert Systems with Applications* 140 (2020), 112905.

[28] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 137–146.

[29] Diederik Kinga, Jimmy Ba Adam, et al. 2015. A method for stochastic optimization. In *International conference on learning representations (ICLR)*, Vol. 5. California;.

[30] Alan Kuhnle, Md Abdul Alim, Xiang Li, Huiling Zhang, and My T Thai. 2018. Multiplex influence maximization in online social networks with heterogeneous diffusion models. *IEEE Transactions on Computational Social Systems* 5, 2 (2018), 418–429.

[31] Sanjay Kumar, Abhishek Mallik, Anavi Khetarpal, and Bhawani Sankar Panda. 2022. Influence maximization in social networks using graph embedding and graph neural network. *Information Sciences* 607 (2022), 1617–1636.

[32] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford large network dataset collection. http://snap.stanford.edu/data.

[33] Hui Li, Mengting Xu, Sourav S Bhowmick, Joty Shafiq Rayhan, Changsheng Sun, and Jiangtao Cui. 2022. PIANO: Influence maximization meets deep reinforcement learning. *IEEE Transactions on Computational Social Systems* 10, 3 (2022), 1288–1300.

[34] Hui Li, Mengting Xu, Sourav S Bhowmick, Changsheng Sun, Zhongyuan Jiang, and Jiangtao Cui. 2019. Disco: Influence maximization meets network embedding and deep learning. *arXiv preprint arXiv:1906.07378* (2019).

[35] Yuchen Li, Ju Fan, Yanhao Wang, and Kian-Lee Tan. 2018. Influence maximization on social graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 30, 10 (2018), 1852–1872.

[36] Zhicheng Liang, Yu Yang, Xiangyu Ke, Xiaokui Xiao, and Yunjun Gao. 2024. A Benchmark Study of Deep-RL Methods for Maximum Coverage Problems over Graphs. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3666–3679.

[37] Su-Chen Lin, Shou-De Lin, and Ming-Syan Chen. 2015. A learning-based framework to handle multi-round multi-party influence maximization on social networks. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. 695–704.

[38] Chen Ling, Junji Jiang, Junxiang Wang, My T Thai, Renhao Xue, James Song, Meikang Qiu, and Liang Zhao. 2023. Deep graph representation learning and optimization for influence maximization. In *International conference on machine learning (ICML)*. PMLR, 21350–21361.

[39] Dandan Liu and Zhaonian Zou. 2023. Gcore: Exploring cross-layer cohesiveness in multi-layer graphs. *Proceedings of the VLDB Endowment* 16, 11 (2023), 3201–3213.

[40] Sahil Manchanda, Akash Mittal, Anuj Dhawan, Sourav Medya, Sayan Ranu, and Ambuj Singh. 2020. GCOMB: learning budget-constrained combinatorial algorithms over billion-sized graphs. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS)*. 20000–20011.

[41] Michael Mitzenmacher and Eli Upfal. 2017. *Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis*. Cambridge university press.

[42] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).

[43] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.

[44] Britannica Money. 2025. Facebook. https://www.britannica.com/money/Facebook

[45] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. 1978. An analysis of approximations for maximizing submodular set functions—I. *Mathematical programming* 14, 1 (1978), 265–294.

[46] Dung T Nguyen, Huiyuan Zhang, Soham Das, My T Thai, and Thang N Dinh. 2013. Least cost influence in multiplex social networks: Model representation and analysis. In *2013 IEEE 13th International Conference on Data Mining (ICDM)*. IEEE, 567–576.

[47] Huyen Nguyen, Hieu Dam, Nguyen Hoang Khoi Do, Cong Tran, and Cuong Pham. 2025. REM: A Scalable Reinforced Multi-Expert Framework for Multiplex Influence Maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 27099–27107.

[48] Elisa Omodei, Manlio De Domenico, and Alex Arenas. 2015. Characterizing interactions in online social networks during exceptional events. *Frontiers in Physics* 3 (2015), 59.

[49] George Panagopoulos, Fragkiskos D Malliaros, and Michalis Vazirgiannis. 2020. Multi-task learning for influence estimation and maximization. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 34, 9 (2020), 4398–4409.
[50] K Venkatakrishna Rao and C Ravindranath Chowdary. 2022. CBIM: Community-based influence maximization in multilayer networks. *Information Sciences* 609 (2022), 578–594.
[51] Martin Riedmiller. 2005. Neural fitted Q iteration–first experiences with a data efficient neural reinforcement learning method. In *European conference on machine learning*. Springer, 317–328.
[52] Shashank Sheshar Singh, Divya Srivastva, Madhushi Verma, and Jagendra Singh. 2022. Influence maximization frameworks, performance, challenges and directions on social network: A theoretical study. *Journal of King Saud University-Computer and Information Sciences* 34, 9 (2022), 7570–7603.
[53] Richard S Sutton, Andrew G Barto, et al. 1998. *Reinforcement learning: An introduction*. Vol. 1. MIT press Cambridge.
[54] Jing Tang, Xueyan Tang, Xiaokui Xiao, and Junsong Yuan. 2018. Online processing algorithms for influence maximization. In *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*. 991–1005.
[55] Jing Tang, Xueyan Tang, and Junsong Yuan. 2017. Profit maximization for viral marketing in online social networks: Algorithms and analysis. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 30, 6 (2017), 1095–1108.
[56] Youze Tang, Yanchen Shi, and Xiaokui Xiao. 2015. Influence maximization in near-linear time: A martingale approach. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1539–1554.
[57] Youze Tang, Xiaokui Xiao, and Yanchen Shi. 2014. Influence maximization: Near-optimal time complexity meets practical efficiency. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. 75–86.
[58] Siyi Teng, Jiadong Xie, Fan Zhang, Can Lu, Juntao Fang, and Kai Wang. 2024. Optimizing Network Resilience via Vertex Anchoring. In *Proceedings of the ACM Web Conference (WWW)*. 606–617.
[59] Shan Tian, Songsong Mo, Liwei Wang, and Zhiyong Peng. 2020. Deep reinforcement learning-based approach to tackle topic-aware influence maximization. *Data Science and Engineering* 5, 1 (2020), 1–11.
[60] Vasu Unnava and Ashwin Aravindakshan. 2021. How does consumer engagement evolve when brands post across multiple social media? *Journal of the Academy of Marketing Science* 49, 5 (2021), 864–881.
[61] Jinghao Wang, Yanping Wu, Xiaoyang Wang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2024. Efficient Influence Minimization via Node Blocking. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2501–2513.
[62] Xiaoning Wang, Yakov Bart, Serguei Netessine, and Lynn Wu. 2025. Impact of Multi-Platform Social Media Strategy on Sales in E-Commerce. *arXiv preprint arXiv:2503.09083* (2025).
[63] Mao Ye, Xingjie Liu, and Wang-Chien Lee. 2012. Exploring social influence for recommendation: a generative model approach. In *Proceedings of the 35th international ACM SIGIR conference on Research and development in information retrieval*. 671–680.
[64] Zirui Yuan, Minglai Shao, and Zhiqian Chen. 2024. Graph bayesian optimization for multiplex influence maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 22475–22483.
[65] Qianyi Zhan, Jiawei Zhang, S Yu Philip, Sherry Emery, and Junyuan Xie. 2016. Discover tipping users for cross network influencing. In *2016 IEEE 17th International Conference on Information Reuse and Integration (IRI)*. IEEE, 67–76.
[66] Huiyuan Zhang, Dung T Nguyen, Huiling Zhang, and My T Thai. 2015. Least cost influence maximization across multiple social networks. *IEEE/ACM Transactions On Networking* 24, 2 (2015), 929–939.
[67] Qingpeng Zhang, Lu Zhong, Siyang Gao, and Xiaoming Li. 2018. Optimizing HIV interventions for multiplex social networks via partition-based random search. *IEEE transactions on cybernetics* 48, 12 (2018), 3411–3419.
[68] Shiqi Zhang, Yiqian Huang, Jiachen Sun, Wenqing Lin, Xiaokui Xiao, and Bo Tang. 2023. Capacity constrained influence maximization in social networks. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 3376–3385.

A Algorithms

A.1 Mlim-Greedy

Algorithm 3 outlines the pseudo-code of Mlim-Greedy. Initially, it initializes an empty seed node set S, and for each node $v \in V$, it sets the influence marginal gain $\Delta(v|S) = 0$ (Lines 1–2). In each iteration (Lines 3–7), Mlim-Greedy computes the influence marginal gain $\Delta(v|S)$ for each node v in $V \setminus S$ (Lines 4–5), selects the node with the maximum marginal gain in expected spread over all layers (Line 6), and adds it to S (Line 7). The process continues until k seed nodes are selected. Finally, the algorithm returns the seed set S (Line 8).

A.2 Supervised GCN Training

Algorithm 4 presents the pseudo-code for the complete GCN training process. The algorithm begins by randomly initializing all learnable parameters, including weight matrices $\{W^{(l)}\}_{l=1}^{K}$ and the final weight vector $\boldsymbol{w}$ (Line 1). The training process then iterates over T epochs (Line 2). In each epoch, the algorithm performs forward propagation by first initializing each node v with its feature vector, $H_v^{(0)} = \boldsymbol{x}_v$ (Line 3). The forward pass consists of an outer loop that iterates over the depth of the model, from 1 to K (Line 4), and an inner loop that traverses all nodes $v \in V$ (Line 5). For each node v, we first aggregate the current representations of its out-neighbors using a MeanPool layer (Line 6). The aggregated vector is then concatenated with v's own representation and passed through a fully connected layer with *ReLU* activation (Line 7). The resulting vector becomes the input for the next iteration of the outer loop. Intuitively, with each iteration of the outer loop, a node progressively incorporates information from neighbors at increasing depths, thereby capturing both local and higher-order structural patterns. During this process, $\{W^{(l)}\}_{l=1}^{K}$ is used in the hidden layers to propagate information across different depths of the model. After the forward propagation, the algorithm generates predicted influence scores by passing the final layer representations through an additional fully connected layer, yielding $\hat{\epsilon}_v^{(\ell)}$ for each node $v \in V_\ell \subseteq V$ (Lines 8-9). The training process then computes the mean squared error loss between predicted and ground-truth scores (Line 10) and updates all parameters using gradient descent (Line 11). This supervised learning process continues until convergence or the maximum number of epochs is reached, ultimately returning the trained parameters $\Theta_S = \{W^{(l)}\}_{l=1}^{K} \cup \{\boldsymbol{w}\}$.

Algorithm 3 Mlim-Greedy

Input: $G(V, E, L)$, k
Output: S
1: $S \leftarrow \emptyset$;
2: $\forall v \in V, \Delta(v|S) \leftarrow 0$;
3: **while** $|S| < k$ **do**
4: **for** $\forall v \in V \setminus S$ **do**
5: $\Delta(v|S) \leftarrow \sum_{i \in L}(\sigma(S \cup \{v\}, G_i) - \sigma(S, G_i))$;
6: $v^* \leftarrow \arg\max_{v \in V \setminus S} \Delta(v|S)$;
7: $S \leftarrow S \cup \{v^*\}$;
8: **Return** S.

Time Complexity. Algorithm 4 has a time complexity of $O(T \times K \times (|E| + |V| \times d \times h))$, where T is the maximal number of training epochs, K is the network depth, $|E|$ and $|V|$ denote the numbers of edges and nodes, respectively, d is the input feature dimension, and h is the hidden layer dimension.

A.3 Q-learning Network Training

The training process is outlined in Algorithm 5. The algorithm begins by randomly initializing the Q-network parameters Θ_Q and the experience replay buffer $\mathcal{J}$ (Lines 1-2). During training, since the Q-network has not yet been sufficiently optimized, we adopt a κ-greedy strategy to balance exploration and exploitation: with probability κ, a random node is chosen from the candidate set C_t;

Algorithm 5 Q-learning Network Training

Input: $G(V, E, L)$, predicted node scores $\hat{\mathcal{E}}$, budget k, experience replay memory $\mathcal{J}$ with capacity N, number of episodes E_p, n-step parameter n, discount factor γ, batch size B
Output: Learned parameter set Θ_Q

1: Initialize Q-network parameters Θ_Q randomly;
2: Initialize experience replay buffer $\mathcal{J} \leftarrow \emptyset$;
3: **for** episode $e_p \leftarrow 1$ to E_p **do**
4: Initialize $S_1 = \emptyset$;
5: $\kappa \leftarrow \max\{0.1, 0.8^{e_p}\}$;
6: **for** step $t \leftarrow 1$ to k **do**
7: $C_t \leftarrow V \setminus S_t$;
8: $v_t \leftarrow \begin{cases} \text{random node } v \in C_t & \text{w.p. } \kappa \\ \text{argmax}_{v \in C_t} Q'_n(S_t, v; \Theta_Q) & \text{w.p. } 1-\kappa \end{cases}$;
9: Compute reward: $r_{t+1} \leftarrow \sigma(S_t \cup \{v_t\}) - \sigma(S_t)$;
10: $S_{t+1} \leftarrow S_t \cup \{v_t\}$;
11: **if** $t \geq n$ **then**
12: $r_{sum} \leftarrow \sum_{i=t-n+1}^{t} r_i$;
13: Add tuple $\langle S_{t-n+1}, v_{t-n+1}, r_{sum}, S_{t+1} \rangle$ to $\mathcal{J}$;
14: **if** $|\mathcal{J}| \geq B$ **then**
15: Sample random batch $\mathcal{B}$ of size B from $\mathcal{J}$;
16: **for** each transition $(S, v, r_{sum}, S') \in \mathcal{B}$ **do**
17: $y \leftarrow r_{sum} + \gamma \cdot \max_{v' \in V \setminus S'} Q'_n(S', v'; \Theta_Q)$;
18: $\mathcal{L}(\Theta_Q) \leftarrow \frac{1}{B} \sum_{(S,v,r_{sum},S') \in \mathcal{B}} (y - Q'_n(S, v; \Theta_Q))^2$;
19: $\Theta_Q \leftarrow \text{Adam}(\Theta_Q, \nabla \mathcal{L}(\Theta_Q))$;
20: **Return** Θ_Q.

Algorithm 4 Supervised GCN Training

Input: $G(V, E, L)$, ground-truth scores $\mathcal{E}$, node features $\boldsymbol{X}$, depth K, learning rate α, max epochs T
Output: Trained parameters $\Theta_S = \{W^{(l)}\}_{l=1}^{K} \cup \{\boldsymbol{w}\}$

1: Randomly initialize weight matrices $\{W^{(l)}\}_{l=1}^{K}$ and weight vector $\boldsymbol{w}$;
2: **for** epoch $t = 1$ to T **do** `//Forward propagation`
3: $H_v^{(0)} \leftarrow \boldsymbol{x}_v, \ \forall v \in V$;
4: **for** each layer $l = 1$ to K **do**
5: **for** each node $v \in V$ **do**
6: $H_{N(v)}^{(l)} \leftarrow \textsc{MeanPool}(\{H_u^{(l-1)} : u \in N_{out}(v)\})$;
7: $H_v^{(l)} \leftarrow ReLU(W^{(l)} \cdot \textsc{Concat}(H_{N(v)}^{(l)}, H_v^{(l-1)}))$;
8: **for** each $\ell \in L$ and $v \in V_\ell$ **do**
9: $\hat{\epsilon}_v^{(\ell)} \leftarrow \boldsymbol{w}^\top \cdot H_v^{(K)}$
10: $\mathcal{L}(\Theta_S) = \frac{1}{|V|} \sum_{\ell \in L} \sum_{v \in V_\ell} (\hat{\epsilon}_v^{(\ell)} - \epsilon_v^{(\ell)})^2$; `//Loss computation`
11: $\Theta_S \leftarrow \Theta_S - \alpha \nabla_{\Theta_S} \mathcal{L}(\Theta_S)$;
12: **Return** $\Theta_S = \{W^{(l)}\}_{l=1}^{K} \cup \{\boldsymbol{w}\}$.

otherwise, the node with the maximum predicted n-step reward is selected (Line 8). The exploration rate κ decays exponentially with the number of episodes e_p (Line 5), allowing the algorithm to increasingly rely on the model's predictions as training progresses. At each step, the algorithm computes the marginal influence gain as the immediate reward (Line 9) and transitions to the next state (Line 10). To incorporate delayed rewards, we apply n-step Q-learning (Lines 11–20): after observing n steps, the cumulative n-step reward is computed and the corresponding experience tuple is stored in the replay memory $\mathcal{J}$ (Lines 12-13). When sufficient experiences are available, a random batch is sampled from $\mathcal{J}$ to compute target values using the Bellman equation (Lines 14–17), and the network parameters Θ_Q are updated via the Adam optimizer to minimize the squared loss (Lines 18–19). For efficient parameter learning, we follow fitted Q-iteration [51], which leverages mini-batch updates from the replay memory instead of sample-by-sample updates. Combined with a neural network function approximator [42], it accelerates convergence and allows the Q-network to effectively learn the best node-selection policy under different states.

Time Complexity. The time complexity of Algorithm 5 is $O(E_p \times k \times (|V| \times T_Q + T_\sigma))$, where E_p is the number of training episodes, k is the budget, $|V|$ is the number of nodes, T_Q is the cost of a single forward pass of the Q-network (with $T_Q = O(|E| + |V| \times h)$, where h is the hidden dimension), and T_σ is the cost of evaluating the influence function.

B Discussion on the Extension to Union Influence Metric $\mathbb{E}[|\bigcup_{i \in L} I(G_i, S)|]$

We discuss the extension of our framework to the union influence metric $\sigma_\cup(S, G) = \mathbb{E}[|\bigcup_{i \in L} I(G_i, S)|]$, which measures the expected number of distinct users activated across all layers.

Estimator Design. The extension does not require fundamental changes to the framework, as the per-layer sampling and layer-wise estimator structure are preserved under the union metric. The key modification is that $\Lambda_R(S, G)$ is redefined to indicate whether S can activate the source node v of M-RR set R in *any* layer, rather than specifically in layer i. To avoid double-counting nodes that appear in multiple layers, we introduce a $1/|L_v|$ reweighting factor, where $|L_v|$ denotes the number of layers in which v appears. The estimator becomes:

$$f^{\mathcal{R}}(S, G) = \sum_{i \in L} f^{\mathcal{R}}(S, G_i) = \sum_{i \in L} \frac{|V_i|}{|\mathcal{R}_i|} \cdot \sum_{R \in \mathcal{R}_i} \frac{1}{|L_v|} \Lambda_R(S, G), \quad (17)$$

which retains the per-layer weighting structure of the original estimator in Eq. (5).

Unbiasedness. Since MRIS samples each node $v \in V_i$ with probability $1/|V_i|$, we have:

$$\begin{aligned} \mathbb{E}\left[f^{\mathcal{R}}(S, G_i)\right] &= \mathbb{E}\left[\frac{|V_i|}{|\mathcal{R}_i|} \cdot \sum_{R \in \mathcal{R}_i} \frac{1}{|L_v|} \Lambda_R(S, G)\right] \\ &= \sum_{v \in V_i} \frac{1}{|L_v|} \Pr[\exists j : v \in I(G_j, S)]. \end{aligned} \quad (18)$$

Summing over all layers:

$$\begin{aligned} \mathbb{E}\left[f^{\mathcal{R}}(S, G)\right] &= \sum_{i \in L} \sum_{v \in V_i} \frac{1}{|L_v|} \Pr[\exists j : v \in I(G_j, S)] \\ &= \sum_{v \in V} \Pr[\exists j : v \in I(G_j, S)] \\ &= \mathbb{E}[|\bigcup_{i \in L} I(G_i, S)|], \end{aligned} \quad (19)$$

where the second equality follows from the fact that each node v appears in $|L_v|$ layers and is thus counted $|L_v|$ times in the summation over layers.

Theoretical Guarantees. Lemma 2 and Theorem 4 remain applicable under this modification. For Lemma 2, since $\frac{1}{|L_v|}\Lambda_R \in [0, 1]$ still satisfies the Chernoff bound conditions, the variance bounds hold with $\sigma(S, G)$ now referring to $\mathbb{E}[|\bigcup_{i \in L} I(G_i, S)|]$. For Theorem 4, the proof framework requires no modification, and the $(1 - 1/e - \epsilon)$ approximation guarantee remains applicable.

C Proofs

C.1 Proof of Theorem 1

Proof. It has been proved that the influence maximization problem in a single-layer graph under the classical IC or LT model is **NP**-hard [28]. We construct a special case of the Mlim problem as follows: (1) consider a multilayer graph that contains only a single layer i, i.e., $G = G_i$; and (2) set the propagation model on G_i to be the specific classical IC or LT model. Under this construction, Mlim reduces to the classical single-layer influence maximization problem. Therefore, Mlim is **NP**-hard. □

C.2 Proof of Theorem 2

Proof. Computing the exact influence spread of a seed node set S on a single-layer graph, i.e., $\sigma(S, G_i)$, under the classical IC or LT model is known to be #**P**-hard [11, 28]. Since the hybrid propagation model generalizes the classical IC/LT models by allowing influence to propagate across multiple layers, any instance of IC/LT on a single layer can be regarded as a special case of the hybrid model. Moreover, the total expected influence spread is defined as the sum over all layers, i.e., $\sigma(S, G) = \sum_{i \in L} \sigma(S, G_i)$. Therefore, computing the exact influence spread $\sigma(S, G)$ under the hybrid propagation model is also #**P**-hard. □

C.3 Proof of Theorem 3

Proof. **(1) Monotonicity:** For any two seed node sets $S_1 \subseteq S_2 \subset V$, the expected influence spread on each layer satisfies $\sigma(S_1, G_i) \leq \sigma(S_2, G_i)$, due to the monotonicity property of the expected influence spread under both IC and LT models [28]. Since the total expected influence spread is the sum over all layers, we have $\sigma(S_1, G) = \sum_{G_i \subseteq G} \sigma(S_1, G_i) \leq \sum_{G_i \subseteq G} \sigma(S_2, G_i) = \sigma(S_2, G)$. This establishes the monotonicity of $\sigma(S, G)$.

(2) Submodularity: Let $S_1 \subseteq S_2 \subset V$ and $v \in V \setminus S_2$. For each layer $i \in L$, $\sigma(S, G_i)$ is submodular, as the submodularity property of the expected influence spread under both IC and LT models [28], i.e., $\sigma(S_1 \cup \{v\}, G_i) - \sigma(S_1, G_i) \geq \sigma(S_2 \cup \{v\}, G_i) - \sigma(S_2, G_i)$. Since the sum of nonnegative submodular functions is also submodular, the overall influence spread $\sigma(S, G) = \sum_{G_i \subseteq G} \sigma(S, G_i)$ is submodular. Thus, $\sigma(S_1 \cup \{v\}, G) - \sigma(S_1, G) \geq \sigma(S_2 \cup \{v\}, G) - \sigma(S_2, G)$. Therefore, $\sigma(S, G)$ is submodular w.r.t. S. □

C.4 PROOF OF LEMMA 1

Proof. Given the monotonicity and submodularity of the Mlim problem established in Section 3, the assurance of approximation follows directly from [45]. □

C.5 PROOF OF LEMMA 2

Proof. Before proving Lemma 2, we first introduce *Chernoff Inequalities* [41] in Lemma 3 as follows.

Lemma 3. (Chernoff Inequalities [41]). *Let X be the sum of k i.i.d. random variables sampled from a distribution on $[0, 1]$ and μ be the mean. Then, for any $\mu > 0$,*

$$\Pr[X - k\mu \geq \mu \cdot k\mu] \leq \exp\left(-\frac{\mu^2}{2+\mu} k\mu\right) \tag{20}$$

$$\Pr[X - k\mu \leq -\mu \cdot k\mu] \leq \exp\left(-\frac{\mu^2}{2} k\mu\right) \tag{21}$$

Consider any solution S to our problem and the optimal solution S^*. Given any set $\mathcal{R}$ of M-RR sets, we extend Lemma 3 with the following concentration bounds:

$$\Pr[f^{\mathcal{R}_2}(S, G_1) - \sigma(S, G_1) \geq \epsilon_1 \cdot \sigma(S, G_1)] \leq \exp\left(-\frac{\epsilon_1^2}{2+\epsilon_1} \frac{|R_1|}{|V_1|} \sigma(S, G_1)\right)$$

$$\Pr[f^{\mathcal{R}_2}(S, G_2) - \sigma(S, G_2) \geq \epsilon_1 \cdot \sigma(S, G_2)] \leq \exp\left(-\frac{\epsilon_1^2}{2+\epsilon_1} \frac{|R_2|}{|V_2|} \sigma(S, G_2)\right)$$

$$\dots$$

$$\Pr[f^{\mathcal{R}_2}(S, G_l) - \sigma(S, G_l) \geq \epsilon_1 \cdot \sigma(S, G_l)] \leq \exp\left(-\frac{\epsilon_1^2}{2+\epsilon_1} \frac{|R_l|}{|V_l|} \sigma(S, G_l)\right)$$

where $|R_i|$ denotes the number of M-RR sets belonging to $\mathcal{R}_2$ sampled in layer i. Then, summing thm yields:

$$\begin{aligned}
&\Pr[f^{\mathcal{R}_2}(S, G) - \sigma(S, G) \geq \epsilon_1 \cdot \sigma(S, G)] \\
&\leq \exp\left(-\frac{\epsilon_1^2}{2+\epsilon_1} \left(\frac{|R_1|}{|V_1|}\sigma(S, G_1) + \frac{|R_2|}{|V_2|}\sigma(S, G_2) + \dots + \frac{|R_l|}{|V_l|}\sigma(S, G_l)\right)\right) \\
&= \exp\left(-\frac{\epsilon_1^2}{2+\epsilon_1} \sum_{i=1, R_i \in \mathcal{R}_2}^{l} \frac{|R_i|}{|V_i|} \sigma(S, G_i)\right)
\end{aligned} \tag{22}$$

where $f^{\mathcal{R}_2}(S, G) = f^{\mathcal{R}_2}(S, G_1) + f^{\mathcal{R}_2}(S, G_2) + \dots + f^{\mathcal{R}_2}(S, G_l)$ and $\sigma(S, G) = \sigma(S, G_1) + \sigma(S, G_2) + \dots + \sigma(S, G_l)$. Similarly, we also have:

$$\begin{aligned}
&\Pr[f^{\mathcal{R}_1}(S^*, G) - \sigma(S^*, G) \leq -\epsilon_2 \cdot \sigma(S^*, G)] \\
&\leq \exp\left(-\frac{\epsilon_2^2}{2} \left(\frac{|R_1|}{|V_1|}\sigma(S^*, G_1) + \frac{|R_2|}{|V_2|}\sigma_2(S^*, G_2) + \dots + \frac{|R_l|}{|V_l|}\sigma_l(S^*, G_l)\right)\right) \\
&= \exp\left(-\frac{\epsilon_2^2}{2} \sum_{i=1, R_i \in \mathcal{R}_1}^{l} \frac{|R_i|}{|V_i|} \sigma_i(S^*, G_i)\right)
\end{aligned} \tag{23}$$

where $|R_i|$ represents the number of M-RR sets belonging to $\mathcal{R}_1$ sampled in layer i. Based on this, in the τ-th iteration, let $\Omega_{1\tau}$ denote the event that Eq. (6) holds, and $\Omega_{2\tau}$ denote the event that Eq. (7) holds. We set ϵ' and $\bar{\epsilon}$ as the solutions to Eq. (22) and Eq. (23) respectively, thus we have following equations:

$$\exp\left(-\frac{(\epsilon')^2}{2+\epsilon'} \sum_{i=1, R_i \in \mathcal{R}_2}^{l} \frac{|R_i|}{|V_i|} \sigma(S, G_i)\right) = \frac{\delta}{5\tau^2}. \tag{24}$$

$$\exp\left(-\frac{(\bar{\epsilon})^2}{2} \sum_{i=1, R_i \in \mathcal{R}_1}^{l} \frac{|R_i|}{|V_i|} \sigma(S, G_i)\right) = \frac{\delta}{5\tau^2}. \tag{25}$$

Then, we have $\Pr[\Omega_{\tau 1}] \geq 1 - \delta/(5\tau^2)$ and $\Pr[\Omega_{\tau 2}] \geq 1 - \delta/(5\tau^2)$.

$$\begin{aligned}
\frac{(\epsilon')^2}{(2+\epsilon')(1+\epsilon')} &= \frac{\ln(5\tau^2/\delta)}{(1+\epsilon') \cdot \sum_{i=1, R_i \in \mathcal{R}_2}^{l} \frac{|R_i|}{|V_i|} \sigma(S, G_i)} \\
&\leq \frac{\ln(5\tau^2/\delta)}{(1+\epsilon') \cdot \min_{R_i \in \mathcal{R}_2} \{\frac{|R_i|}{|V_i|}\} \cdot \sum_i^l \sigma(S, G_i)} \\
&= \frac{\ln(5\tau^2/\delta)}{(1+\epsilon') \cdot \min_{R_i \in \mathcal{R}_2} \{\frac{|R_i|}{|V_i|}\} \cdot \sigma(S, G)} \\
&\leq \frac{\ln(5\tau^2/\delta)}{\min_{R_i \in \mathcal{R}_2} \{\frac{|R_i|}{|V_i|}\} \cdot f^{\mathcal{R}_2}(S, G)}
\end{aligned} \tag{26}$$

$$\begin{aligned}\frac{\bar{\epsilon}^2}{2\,(1+\epsilon_1)} &= \frac{\ln\left(5\tau^2/\delta\right)}{(1+\epsilon_1)\cdot\sum_{i=1,R_i\in\mathcal{R}_1}^{l}\frac{|R_i|}{|V_i|}\sigma(S^*,G_i)}\\ &\le \frac{\ln\left(5\tau^2/\delta\right)}{(1+\epsilon_1)\cdot\min_{R_i\in\mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\}\cdot\sum_{i}^{l}\sigma(S^*,G_i)}\\ &= \frac{\ln\left(5\tau^2/\delta\right)}{(1+\epsilon_1)\cdot\min_{R_i\in\mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\}\cdot\sigma(S^*,G)}\\ &\le \frac{\ln\left(5\tau^2/\delta\right)}{(1+\epsilon_1)\cdot\min_{R_i\in\mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\}\cdot\sigma(S,G)}\\ &\le \frac{\ln\left(5\tau^2/\delta\right)}{\min_{R_i\in\mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\}\cdot f^{\mathcal{R}_2}(S,G)}\end{aligned} \tag{27}$$

We have $\Pr[\Omega_{\tau 1}] \ge 1-\delta/(5\tau^2)$, $\Pr[\Omega_{\tau 2}|\Omega_{\tau 1}] \ge 1-\delta/(5\tau^2)$. Thus, $\Pr[\Omega_{\tau 2}\cap\Omega_{\tau 1}] = \Pr[\Omega_{\tau 2}|\Omega_{\tau 1}]\cdot\Pr[\Omega_{\tau 1}] \ge 1-2\delta/(5\tau^2)$. For all iterations, we have

$$\begin{aligned}\Pr\left[\bigcap_{\tau=1}^{\infty}\Omega_{1\tau}\bigcap_{\tau=1}^{\infty}\Omega_{2\tau}\right] &\ge \prod_{\tau=1}^{\infty}\Pr\left[\Omega_{1\tau}\cap\Omega_{2\tau}\right] \ge \prod_{\tau=1}^{\infty}\left(1-\frac{2\delta}{5\tau^2}\right)\\ &\ge 1-\sum_{\tau=1}^{\infty}\frac{2\delta}{5\tau^2} \ge 1-\frac{\pi^2\delta}{15} \ge 1-\frac{2\delta}{3}.\end{aligned} \tag{28}$$

The details of proof are similar in spirit to those in [25]. □

C.6 PROOF OF THEOREM 4

Proof. With the above conclusions, we prove the Theorem 4. We consider two cases that depend on whether Line 9 in Algorithm 1 is satisfied.

Case 1: Line 9 is satisfied. Then in the last iteration, we have

$$\mu \le \frac{1-1/e}{1-1/e-\epsilon}\cdot\frac{1-\epsilon_2}{1+\epsilon_1} \tag{29}$$

where $\epsilon_1,\epsilon_2\in(0,1)$ and $\mu>0$. Suppose that both Eq. (6) and Eq. (7) hold. Then,

$$\begin{aligned}f^{\mathcal{R}_1}(S,G) &\ge (1-1/e)f^{\mathcal{R}_1}(S^*,G)\\ &\ge (1-1/e)(1-\epsilon_2)\sigma(S^*,G)\end{aligned} \tag{30}$$

Afterwards, via Eq. (6) we have:

$$f^{\mathcal{R}_1}(S,G) = \mu f^{\mathcal{R}_2}(S,G) \le \mu(1+\epsilon_1)\sigma(S,G) \tag{31}$$

Finally, we have

$$\begin{aligned}\sigma(S,G) \ge \frac{1}{\mu(1+\epsilon_1)}f^{\mathcal{R}_1}(S,G) &\ge \frac{(1-1/e)(1-\epsilon_2)}{\mu(1+\epsilon_1)}\cdot\sigma(S^*,G)\\ &\ge (1-1/e-\epsilon)\cdot\sigma(S^*,G)\end{aligned} \tag{32}$$

where the first inequality is from Eq. (31), the second inequality is from Eq. (30), and the third inequality is from Eq. (29). By Lemma 2, when Line 9 in Algorithm 1 is satisfied, with probability at least $1-\frac{2\delta}{3}$, Theorem 4 holds.

Case 2: Line 9 is not satisfied. Then we have

$$\min_{R_i\in\mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\} \ge (8+2\epsilon)(1+\epsilon_1)\frac{\ln\frac{6}{\delta}+|V|\ln 2}{\epsilon^2\cdot f^{\mathcal{R}_2}(S,G)}, \tag{33}$$

when Algorithm 1 terminates. Note that when $\bigcap_\tau\Omega_{1\tau}$ occurs, it implies that $f^{\mathcal{R}_2}(S,G)\le(1+\epsilon_1)\sigma(S,G)\le(1+\epsilon_1)\sigma(S^*,G)$. Then

$$\min_{R_i\in\mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\} \ge (8+2\epsilon)\frac{\ln\frac{6}{\delta}+|V|\ln 2}{\epsilon^2\cdot\sigma(S^*,G)}, \tag{34}$$

when Algorithm 1 terminates. Then by Lemma 3, let $x=\frac{\epsilon\sigma(S^*,G)}{2\sigma(S,G)}$ for any $S\subseteq V$,

$$\begin{aligned}&\Pr[f^{\mathcal{R}_1}(S,G)-\sigma(S,G)\ge\frac{\epsilon}{2}\cdot\sigma(S^*,G)]\\ &\le \exp\left(-\frac{x^2}{2+x}\cdot\sum_{i=1,R_i\in\mathcal{R}_1}^{l}\frac{|R_i|}{|V_i|}\cdot\sigma(S,G_i)\right)\\ &\le \exp\left(-\frac{x^2}{2+x}\cdot\min_{R_i\in\mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\}\cdot\sigma(S,G)\right)\\ &\le \exp\left(-\frac{\epsilon^2}{8+2\epsilon}\cdot\min_{R_i\in\mathcal{R}_1}\{\frac{|R_i|}{|V_i|}\}\cdot\sigma(S^*,G)\right)\\ &\le \frac{\delta}{6\cdot 2^{|V|}},\end{aligned} \tag{35}$$

where the second inequality is due to the fact that if $\sigma(S,G)=\sigma(S^*,G)$, the right side of the first inequality achieves its maximum. Similarly, we also have $\Pr[f^{\mathcal{R}_1}(S,G)-\sigma(S,G)\le(-\frac{\epsilon}{2})\cdot\sigma(S^*,G)]\le\frac{\delta}{6\cdot 2^{|V|}}$. Thus, we have $\Pr[|f^{\mathcal{R}_1}(S,G)-\sigma(S,G)|\le\frac{\epsilon}{2}\cdot\sigma(S^*,G),\forall S\subseteq V]\ge 1-\frac{\delta}{3}$. When $|f^{\mathcal{R}_1}(S,G)-\sigma(S,G)|\le\frac{\epsilon}{2}\sigma(S^*,G)$ for all $S\subset V$, we have

$$f^{\mathcal{R}_1}(S^*,G) \ge (1-\frac{\epsilon}{2})\sigma(S^*,G)\,, \tag{36}$$

$$f^{\mathcal{R}_1}(S,G) \le \sigma(S,G)+\frac{\epsilon}{2}\sigma(S^*,G). \tag{37}$$

Based on the above results, when the event $\bigcap_\tau\Omega_{1\tau}$ occurs, we have

$$\begin{aligned}\sigma(S) &\ge f^{\mathcal{R}_1}(S,G)-\frac{\epsilon}{2}\sigma(S^*,G)\\ &\ge (1-1/e)f^{\mathcal{R}_1}(S^*,G)-\frac{\epsilon}{2}\sigma(S^*,G)\\ &\ge (1-1/e)(1-\frac{\epsilon}{2})\sigma(S^*,G)-\frac{\epsilon}{2}\sigma(S^*,G)\\ &= (1-1/e-\frac{\epsilon}{2}+\frac{\epsilon}{2e})\sigma(S^*,G)-\frac{\epsilon}{2}\Sigma(S^*,G)\\ &= (1-1/e-\epsilon+\frac{\epsilon}{2e})\sigma(S^*,G)\\ &\ge (1-1/e-\epsilon)\cdot\sigma(S^*,G)\end{aligned} \tag{38}$$

According to Eq. (28), the event $\bigcap_i\Omega_{1i}$ happens with probability at least $1-\frac{2\delta}{3}$. Hence, when Line 9 is not satisfied, with probability at least $1-\frac{2\delta}{3}-\frac{\delta}{3}\ge 1-\delta$, we have

$$\sigma(S,G) \ge (1-1/e-\epsilon)\cdot\sigma(S^*,G). \tag{39}$$

Finally, combine **Case 1** and **Case 2**, the Theorem 4 is demonstrated. □

C.7 PROOF OF THEOREM 5

Proof. The time complexity of Algorithm 1 is primarily determined by the cost of M-RR set generation. In Algorithm 1, the total number of M-RR set generated is at most $O(|V|\cdot(\ln 1/\delta+\ln|V|)/\epsilon^2)$. The expected time to generate a random M-RR set is bounded by $\frac{|E|}{|V|}\cdot\sigma(\{v^*\})$ [57]. Thus, the expected time complexity of Algorithm 1 is $O(\frac{(\ln 1/\delta+\ln|V|)\cdot|E|\cdot\sigma(\{v^*\})}{\epsilon^2})$. □